

MASTER PRODUCT REQUIREMENTS &
ENGINEERING SPECIFICATION

Multi-Tenant Sales, Distribution & Marketing SaaS

The complete build contract for a production-grade FMCG and tobacco route-to-market platform: seventeen domain modules, every dashboard and CRUD surface, a 54-table database design with full SQL DDL, the complete API catalog, and the security, testing and DevOps envelope required to reach production.

Product & Engineering Requirements - Version 1.0

Baseline for construction · Prepared for the AI build agent and engineering leadership

SCOPE

A to Production

MODULES

17 Domain Modules

DESIGN

Premium Dual-Theme UI

OCTOBER 2026 · REFERENCE MARKET: BANGLADESH (BDT)

Table of Contents

1 Document Guide & Reading Protocol	5
1.1 Document Control	5
1.2 How the Build Agent Must Read This Specification	5
1.3 Requirement Language and Conventions	6
1.4 The One-Line Product Definition	6
1.5 What "Production Level" Means in This Document	6
2 Business Context & Domain Glossary	7
2.1 The Reference Distribution Chain	7
2.2 The Reporting Hierarchy (an Example, Never a Rule)	7
2.3 Industries and Tenant Profiles	8
2.4 Domain Glossary	8
3 Platform Architecture & Multi-Tenancy	9
3.1 Layered Architecture	9
3.2 Multi-Tenancy Model Decision	10
3.3 Tenant Isolation Contract (MUST Set)	11
3.4 Module Map and Boundaries	11
3.5 Cross-Cutting Concerns	12
4 Authentication, RBAC & Data Scope	12
4.1 Authentication Requirements	13
4.2 The Four-Part Authorization Model	13
4.3 Default Role Catalog	14
4.4 Scope Resolution Rules	14
4.5 Worked Example	15
5 Organization Model	15
5.1 Structural Entities	15
5.2 Two Contrasting Configurations	16
5.3 Binding Rules (MUST Set)	16
5.4 Module Deliverables and Acceptance Criteria	17
6 Master Data I: Employee, User & Geography	17
6.1 Employee & User Management	17
6.2 Geography: Two Systems, One Purpose	18
6.3 API Surface (this module)	19
7 Master Data II: Distributor, Outlet & Product	20

7.1 Distributor Management	20
7.2 Outlet Management & Segmentation	21
7.3 Product Management	21
7.4 Module API Surface	22
8 Field Force: Routes, Visits & CSR Mobile Flow	22
8.1 Route / Beat Management	23
8.2 Visit Lifecycle	23
8.3 CSR Mobile Experience Requirements	24
8.4 Offline Sync & Conflict Resolution (MUST Set)	24
9 Sales Operations: Orders, Transactions & Performance	25
9.1 Sales Order Lifecycle	25
9.2 Sales Transactions	26
9.3 Performance Metrics (definitions are contractual)	26
9.4 CRUD Matrix & API Surface	26
10 Target Management & Allocation Cascade	27
10.1 Target Types & Periods	27
10.2 The Allocation Cascade	28
10.3 Achievement Computation	28
10.4 CRUD Matrix & API Surface	29
11 Marketing: Campaigns, Trade Marketing & Competitors	29
11.1 Campaign Management	29
11.2 Trade Marketing & POSM Execution	30
11.3 Competitor Intelligence	31
11.4 CRUD Matrix & API Surface	31
12 Inventory, Collection & Approval Engine	32
12.1 Inventory Visibility	32
12.2 Collection Management	32
12.3 API Surface	33
12.4 Configurable Approval Workflow Engine	33
13 Platform Services: Notifications, Files & Audit	34
13.1 Notification System	34
13.2 File & Attachment Management	35
13.3 Audit Logging	35
14 Dashboard Specifications (All Roles)	36
14.1 Super Admin (Platform Plane)	36

14.2 CEO	36
14.3 Head of Sales	36
14.4 Regional Manager / ASM / TSO (ladder pattern)	37
14.5 Supervisor	37
14.6 CSR Mobile Dashboard	37
14.7 Marketing Dashboard	37
14.8 Cross-Cutting Dashboard Rules	38
15 Reporting Engine & Report Catalog	38
15.1 Engine Requirements	38
15.2 Report Catalog (v1 MUST ship all of these)	38
16 UI/UX Design System: Premium Dual-Theme	39
16.1 Theme Contract (MUST Set)	40
16.2 Design Tokens	40
16.3 Component Library Inventory	41
16.4 Responsive & Device Rules	41
16.5 Accessibility & Localization	41
17 Frontend Page & Screen Inventory	42
18 Database Design Standards & ERD	43
18.1 Naming & Typing Conventions	43
18.2 Standard Column Block	44
18.3 Domain-Level ERD - Master Data	44
18.4 Domain-Level ERD - Operational Domains	45
18.5 Table Inventory by Module	45
19 Complete SQL DDL	46
19.1 Reading the Listings	46
19.2 Platform & Tenancy	46
19.3 Identity & Access	48
19.4 Organization & Geography	49
19.5 Distribution & Outlets	51
19.6 Product Catalog	52
19.7 Field Force	54
19.8 Sales, Targets & Marketing	55
19.9 Inventory, Finance & System Services	59
19.10 Cross-Cutting FK Pass & Tamper Enforcement	62
20 API Standards & Complete Endpoint Catalog	63
20.1 API Standards (MUST Set)	63

20.2 Platform & Tenancy	64
20.3 Identity, Organization & Geography	64
20.4 Distributors, Outlets & Products	65
20.5 Field Force & Sync	65
20.6 Sales, Targets, Marketing, Inventory, Finance	66
20.7 Workflow, Notifications, Reports, Files, Audit	67
21 Security & Non-Functional Requirements	68
21.1 Security Controls	68
21.2 Threats to Controls Mapping	68
21.3 Non-Functional Targets	69
22 Testing Strategy & Tenant Isolation Matrix	69
22.1 Test Pyramid & Required Types	69
22.2 Tenant Isolation Test Matrix (zero-tolerance suite)	70
22.3 Authorization Test Cases	70
22.4 Test Data & Environment Strategy	71
23 Definition of Done, Build Order & Phase Gates	71
23.1 Definition of Done (per module)	71
23.2 Master Build Order	72
23.3 Module Development Protocol (for the AI agent)	73
24 Infrastructure, DevOps & Production Readiness	73
24.1 Deployment Topology	73
24.2 CI/CD Pipeline	74
24.3 Observability & Recovery	75
Appendices	75
A Appendix A - Glossary (consolidated)	75
B Appendix B - Default Permission Catalog	75
C Appendix C - Seed Data Requirements	76
D Appendix D - Explicitly Deferred Scope	77

CHAPTER 01

Document Guide & Reading Protocol

This document is the single, authoritative build contract for a multi-tenant Sales, Distribution and Marketing SaaS platform aimed at FMCG, tobacco, beverage, food and pharmaceutical distribution businesses. It is written for an autonomous AI build agent: every chapter is phrased as an implementable requirement with explicit entities, business rules, CRUD boundaries, dashboards, API contracts, database definitions and acceptance gates. Nothing in this specification is optional unless it is explicitly marked as such, and nothing may be silently renamed, merged or dropped during implementation.

1.1 Document Control

Attribute	Value
Document title	Master Product Requirements & Engineering Specification
Product	Multi-Tenant Sales, Distribution & Marketing Management SaaS
Version / status	1.0 - Baseline for construction
Primary audience	AI build agent (autonomous implementation), reviewed by human architects
Reference market	Bangladesh-first (BDT currency, Chattogram examples), internationally portable
Design mandate	Stack-agnostic architecture; premium dual-theme (light + dark) UI with toggle
Scope	All modules, dashboards, CRUD, database design, APIs, security, testing, DevOps, delivery phases

Table 1.1 - Document control record

1.2 How the Build Agent Must Read This Specification

The chapters are ordered deliberately. Chapters 1-5 define the invariants: architecture, tenancy, identity, authorization and organization. Chapters 6-13 define the functional modules in dependency order. Chapters 14-20 define the experience and data layers: dashboards, reporting, the design system, page inventory, the entity-relationship model and complete SQL DDL. Chapters 21-24 define the quality envelope: security, testing, definition of done, build order and production operations. The appendices carry the glossary, the full permission catalog and seed data requirements.

- 1. **Read chapters 3, 4 and 19 before writing any code.** The tenancy model, the authorization model and the database naming rules contaminate every table, every query and every endpoint; getting them wrong invalidates all later work.
- 2. **Implement in the chapter order of the Master Build Order (Chapter 23),** not in the reading order. The build order resolves dependencies between modules; the reading order resolves concepts.

3. **Treat every MUST as a hard gate.** A module that misses a MUST is not complete regardless of how much of it works. SHOULD items may be deferred only with a written rationale in the worklog.
4. **Never implement the whole application in one pass.** Work module by module: inspect the existing architecture and database, plan the delta, implement, test, verify tenant isolation and authorization, then proceed.
5. **Before modifying shared infrastructure, identify all affected modules** and re-run their regression tests. Never silently change existing business rules.

1.3 Requirement Language and Conventions

Keyword	Meaning	Failure handling
MUST / MUST NOT	Absolute requirement of the platform.	Build fails acceptance; no merge.
MUST NOT (security)	Absolute prohibition, especially for tenant isolation.	Critical severity defect.
SHOULD	Strong recommendation; deviation needs a documented reason.	Documented in worklog with rationale.
MAY	Genuinely optional capability; implement if time permits.	Track as backlog item.
Field notation	entity.field written in monospace refers to the DDL in Chapter 19.	Names are contractual; do not rename.
Permission codes	module.action strings from Appendix B; also enforced in code.	Missing permission check = defect.

Table 1.2 - Requirement keywords (RFC-2119 style) and notation

1.4 The One-Line Product Definition

Product definition

A single platform where a platform operator onboards independent tenant companies; each tenant models its own organization, sales geography, distributors, outlets and products; its field force (CSRs) executes daily routes with offline-capable visits; orders, targets, marketing campaigns, inventory and collections are tracked against that geography; and every role - from CEO to CSR - sees exactly the slice of data its scope allows, under a premium light or dark interface it can toggle at will.

1.5 What "Production Level" Means in This Document

Production level is defined by the Definition of Done in Chapter 23: every module ships with its database layer, backend authorization, tenant isolation proof, validation, frontend states (loading, error, empty), pagination, search and filter, audit coverage and tests. It also means the non-functional envelope of Chapter 21 is met: sub-second interactive responses at reference scale, tenant-isolated exports, encrypted secrets, rate limiting and a restore-tested backup regime. Anything that would be embarrassing to show a paying tenant on day one is out of scope for "done".

CHAPTER 02

Business Context & Domain Glossary

The platform digitizes the classic South Asian FMCG/tobacco route-to-market model: a manufacturer sells through distributors, distributors serve retail outlets, and a supervised field force visits those outlets on fixed beats to take orders, check stock, execute trade marketing and collect competitive intelligence. Management cascades revenue targets down the same geography the field force walks, then measures achievement from the orders the field force actually books. All currency examples use Bangladeshi Taka (BDT, symbol Taka) and Chattogram geography, but no currency, geography or hierarchy is hard-coded anywhere in the platform.

2.1 The Reference Distribution Chain

The reference business model flows value from the company through intermediaries to the consumer, and flows information - orders, stock, money, competitor signals - back up. The platform is the system of record for that two-way flow. It must therefore model three parallel structures that happen to overlap: the reporting organization (people), the sales geography (places) and the trade network (distributors and outlets).

- **Trade flow:** Company produces - Distributor stocks - CSR books orders at outlets - Distributor fulfills - Retailer sells to consumer.
- **Information flow up:** orders, visit data, outlet stock, competitor activity, collections, market photos.
- **Information flow down:** targets, route plans, campaigns, POSM instructions, price lists, approvals.

2.2 The Reporting Hierarchy (an Example, Never a Rule)

The first reference tenant, a tobacco company, manages performance through the hierarchy below. The platform must reproduce this chain for reporting and target cascades without encoding it: every level is simply a position in a configured organization attached to a node of the configured sales geography.

Level	Role (example)	Geographic scope (example)	Primary question they ask the system
1	CEO	TENANT - whole company	Are we hitting the national number? Which region is slipping?
2	Commercial Director	TENANT	How does channel and product mix look this month?
3	Head of Sales	TENANT	Region ranking, target gaps, CSR productivity
4	Regional Manager	REGION - Chattogram	Which area is behind? Which TSO needs help?
5	Area Manager (ASM)	AREA - South Chattogram	Territory coverage, distributor health
6	Territory Officer (TSO)	TERRITORY - Panchlaish	CSR performance, outlet coverage, availability

Level	Role (example)	Geographic scope (example)	Primary question they ask the system
7	Field Sales Supervisor	ROUTE set	Team attendance, visits, orders
8	CSR / Sales Rep	ASSIGNED ROUTES	Today's route, outlets, orders, achievement

Table 2.1 - Example reporting hierarchy of reference Tenant A (ABC Tobacco)

2.3 Industries and Tenant Profiles

Industry	Typical depth	Module emphasis	Notes
Tobacco	8+ levels, dense route network	Targets, field force, trade marketing, competitor tracking	Reference tenant; regulated marketing content
FMCG / Food	6-8 levels	Sales, distribution, outlet coverage	High SKU counts, pack sizes matter
Beverage	4-6 levels	Cold-chain visibility, POSM, seasonality	Chilled vs ambient stock distinction
Pharma distribution	5-7 levels	Compliance, batch visibility, approvals	Stricter audit expectations
General distribution	Configurable	Sales, collection, inventory	Lowest common denominator

Table 2.2 - Target industries and how module emphasis shifts

2.4 Domain Glossary

These terms are used with fixed meanings throughout the specification and in the UI copy. The build agent MUST use them consistently in code names, labels and reports so that field users recognize their own vocabulary.

Term	Definition
Tenant	An independent company subscribed to the platform; the boundary of all data isolation.
CSR	Consumer Sales Representative - the field user who walks routes, visits outlets, books orders.
TSO	Territory Sales Officer - manages all CSRs and routes inside one territory.
ASM	Area Sales Manager - manages several territories inside one area.
Route / Beat	An ordered set of outlets a CSR covers in one working day.
Outlet	A retail shop (or wholesale/super shop point) where products are sold to consumers.
Productive Call	An outlet visit that produced at least one order line.
Strike Rate	Productive calls divided by total visited outlets, expressed as a percentage.
Coverage	Outlets visited at least once in the period divided by outlets planned.
POSM	Point-of-sale materials: posters, shelf strips, display units, danglers, signage.
Trade Marketing	Outlet-level execution of brand visibility: displays, POSM placement, activations.

Term	Definition
Target Cascade	Top-down allocation of a numeric target through scope nodes to individuals.
Primary Sales	Company to distributor sales (invoiced volume).
Secondary Sales	Distributor to outlet sales (what CSR orders represent).
Fulfillment	Distributor-side picking, packing and delivery of an approved order.
Collection	Cash/cheque/digital payment collected by CSR or distributor against an invoice.
Offline Sync	Queued local writes on the CSR device replayed to the server when connectivity returns.
Feature Flag	Per-tenant switch that enables or disables a module at runtime.

Table 2.3 - Domain glossary (also reproduced in Appendix A)

CHAPTER 03

Platform Architecture & Multi-Tenancy

The platform is a modular, layered system with one non-negotiable property: tenant isolation enforced at every layer, never merely filtered in the UI. The stack is deliberately unspecified - any mature web stack with a relational database, a cache, a queue and server-rendered or SPA frontends can implement this design. The architecture chapter fixes the layer responsibilities, the tenancy model and the module boundaries; it does not fix frameworks.

3.1 Layered Architecture

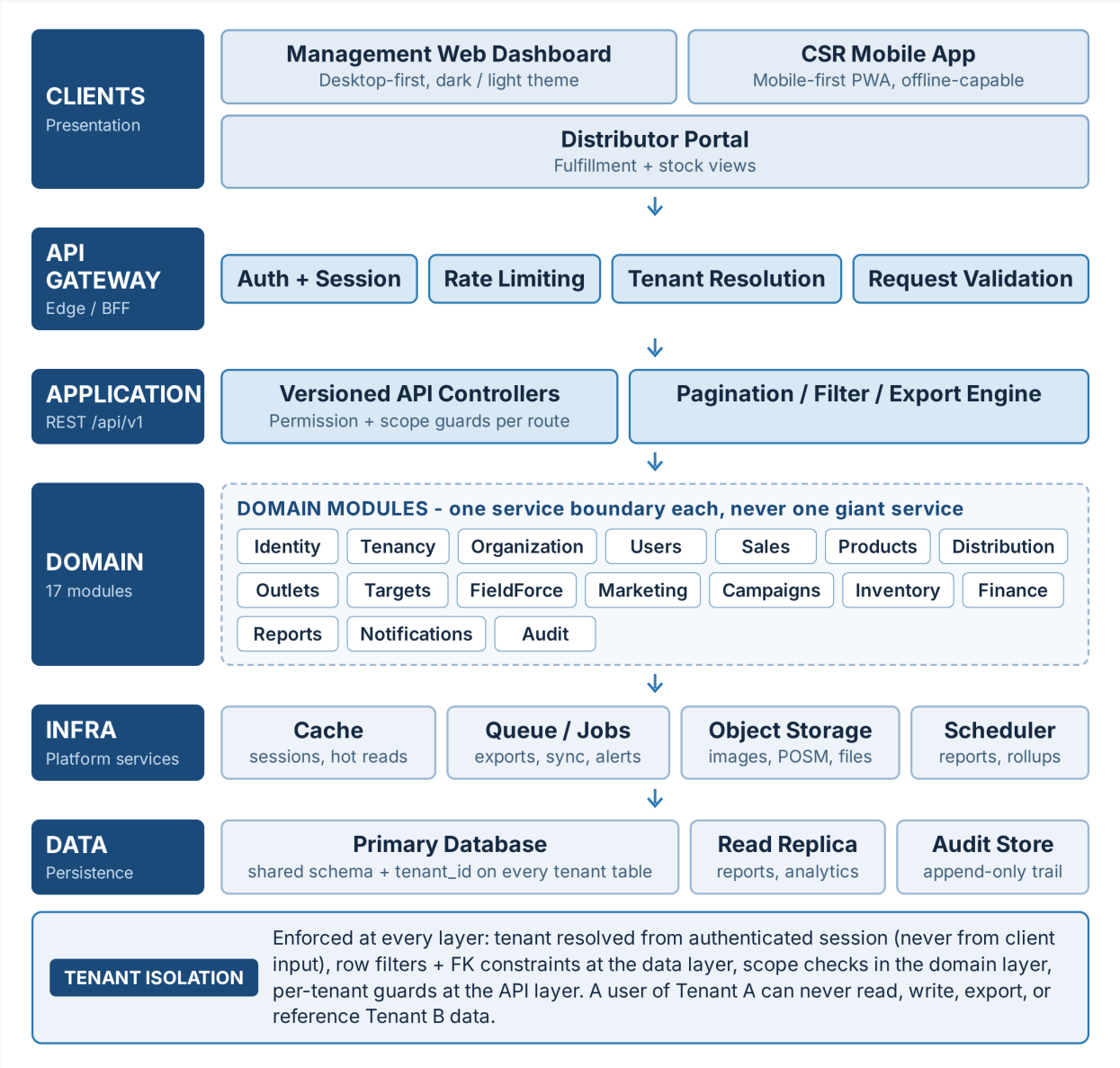


Figure 3.1 - Layered architecture with the seventeen domain modules and the tenant isolation contract

Each layer has a single responsibility. Clients render experience and never make authorization decisions. The gateway authenticates, rate-limits and resolves the tenant from the session. The application layer exposes versioned REST endpoints with per-route permission and scope guards. The domain layer contains the seventeen business modules as separate service boundaries with their own transactional logic. Infrastructure services (cache, queue, storage, scheduler) are consumed through interfaces so implementations can be swapped. The data layer is a relational database in shared-schema multi-tenancy plus an append-only audit store.

3.2 Multi-Tenancy Model Decision

Three tenancy models were evaluated. The shared-schema model with a mandatory tenant_id discriminator was selected because the target market expects hundreds of small-to-mid tenants rather than a handful of giants, operational cost must stay low, and cross-tenant platform analytics (for the platform operator) are

desirable. The trade-offs and mandatory compensations are listed below.

Model	Isolation strength	Ops cost	Verdict
Database-per-tenant	Strongest; easy restore per tenant	High; migration fan-out, connection pressure	Rejected as default; keep as future enterprise option
Schema-per-tenant	Strong; moderate migration cost	Medium-high; catalog bloat at scale	Rejected
Shared schema + tenant_id (chosen)	Strong WHEN enforced per the rules below	Low; single migration line	CHOSEN - with the compensating controls in 3.3

Table 3.1 - Tenancy model evaluation

3.3 Tenant Isolation Contract (MUST Set)

Shared-schema tenancy is only safe when the following controls are all present. They are individually testable, and Chapter 22 turns each row into a mandatory test case. A build that skips any of these controls MUST NOT be considered tenant-isolated, regardless of UI behavior.

1. **Server-derived tenant context.** The tenant of a request is derived from the authenticated session or API token server-side. Any tenant identifier supplied by the client MUST be ignored and, in debug contexts, logged as a suspicion signal.
2. **Central query scope.** Every repository/query path for tenant-owned tables applies the tenant filter automatically at the data-access layer. Ad-hoc queries that bypass the central scope helper MUST fail code review and are prohibited.
3. **Database constraints.** Every tenant-owned table has tenant_id NOT NULL with an index leading on tenant_id; composite FKs (tenant_id, id) from child to parent tables make cross-tenant references physically impossible where supported.
4. **Scope-aware reads.** On top of tenant filtering, the geographic scope of the user further restricts rows (Chapter 4). Export endpoints apply the identical scope filter - exports are a favorite leak path.
5. **File isolation.** Attachment keys are prefixed with tenant id and download URLs are short-lived, signed, and re-authorized server-side per request.
6. **Audit on access.** Cross-module and admin-style access is auditable; audit rows themselves are tenant-scoped except for platform-level audit of tenant administration actions.

3.4 Module Map and Boundaries

Module	Owns	Must not own
Identity	Users, employees, authentication, sessions, password policy	Roles definition (owns assignment only)
Tenancy	Tenants, provisioning, plans binding, feature flags, tenant settings	User management inside a tenant
Organization	Organizations, business units, departments, positions	Sales geography

Module	Owns	Must not own
Geography	Sales regions, areas, territories, routes + govt reference geography	Outlet registration
Products	Categories, brands, products, SKUs, price lists	Stock quantities
Distribution	Distributors, distributor users, distributor stock	Order lifecycle
Outlets	Outlets, segmentation, outlet assignments	Routes
FieldForce	Route plans, visits, activities, attendance	Order state machine (fires events to Sales)
Sales	Sales orders, order items, transactions, order workflow	Invoices (raises events to Finance)
Targets	Target definitions, allocation cascade, achievement rollups	Sales data (reads transactions)
Marketing	Campaigns, trade executions, POSM, competitor records	Product master
Inventory	Stock visibility: distributor, outlet, CSR-reported; movements	Warehouse ERP functions
Finance	Invoices, payments, outstanding, collections	Order approval flow
Reports	Report engine, saved reports, exports	Module-specific business logic
Notifications	Event fan-out to in-app/email/SMS/push providers	Business rules that raise events
Workflow	Generic approval workflow engine, approver resolution	Domain-specific approval rules (supplied as config)
Audit	Audit log write API and viewer; tamper-evident trail	Business data

Table 3.2 - The seventeen domain modules and their ownership boundaries

3.5 Cross-Cutting Concerns

Six concerns cut across every module and are implemented once, centrally: authentication and session management; permission and scope checking; tenant context propagation; validation with localized error messages; audit event emission; and pagination/sort/filter for all list endpoints (server-side only, never client-side over large sets). Notification emission is likewise centralized: domain modules publish events, the Notifications module fans them out. Money handling **MUST** use integer minor units (poisha) or a fixed-scale decimal type - never floating point.

CHAPTER 04

Authentication, RBAC & Data Scope

Authorization in this platform is the composition of four independent answers: who the person is (authentication and user), where they sit (position in the organization), what they may do (role granting granular permissions) and which slice of the world they may do it to (geographic scope). Conflating these four is the single most common design error in this domain and is explicitly forbidden. A CSR is not a permission level; it is a position that typically holds a CSR role with a route-level scope.

4.1 Authentication Requirements

- Credential login with email or phone + password; password policy: minimum 8 characters, entropy check against common lists, secure hash (bcrypt/argon2id).
- Session fixation protection, idle timeout (tenant-configurable, default 30 minutes for management web, 12 hours for CSR mobile with refresh token rotation).
- Account lockout with exponential backoff after repeated failures; lockout events audited and notified to tenant admins.
- Password reset via signed, single-use, short-lived tokens; reset MUST NOT reveal whether an email exists.
- Optional MFA (TOTP) per user, tenant-enforceable via settings - the data model and login flow MUST be MFA-ready at v1.
- API tokens for integrations: per-token scope and permission ceiling, revocable, audited.

4.2 The Four-Part Authorization Model

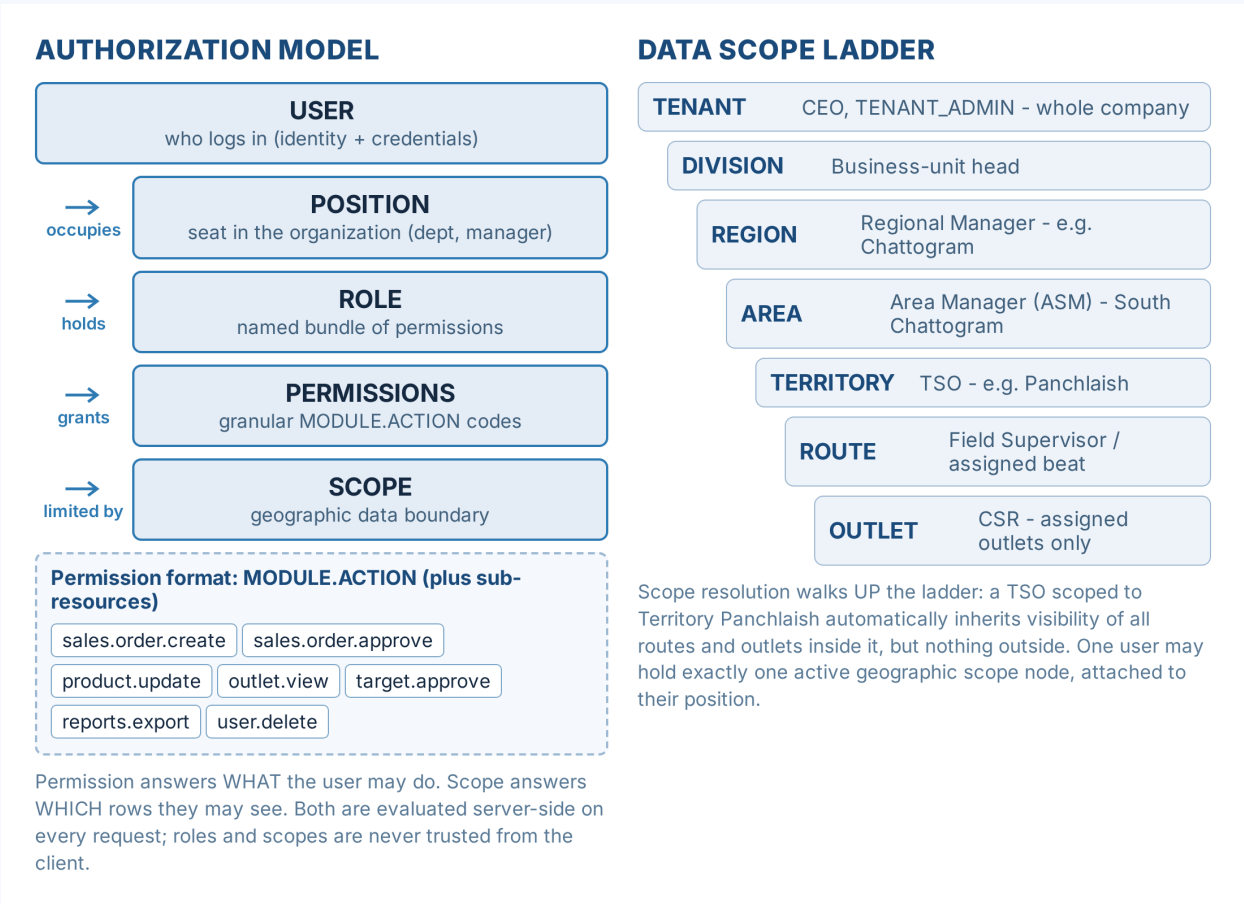


Figure 4.1 - Authorization model (user-position-role-permission-scope) and the data scope ladder

Permissions are strings of the form module.action, with optional sub-resources (sales.order.approve). Roles are named bundles of permissions, tenant-editable from a seeded library. A user holds roles through their

position or directly; effective permissions are the union. Scope is a single active geographic node per user attached through user_scope; the platform derives the full visible subtree from that node. Both permission checks and scope filters are evaluated server-side on every request without exception.

4.3 Default Role Catalog

Family	Roles	Typical scope
Platform	SUPER_ADMIN, PLATFORM_ADMIN, PLATFORM_SUPPORT	Platform plane only; tenant data access only via explicit, audited support grant
Tenant executive	TENANT_ADMIN, CEO, COMMERCIAL_DIRECTOR	TENANT
Sales	HEAD_OF_SALES, REGIONAL_MANAGER, AREA_MANAGER, TERRITORY_OFFICER, FIELD_SUPERVISOR, CSR	TENANT / REGION / AREA / TERRITORY / ROUTE set / ASSIGNED ROUTES
Marketing	HEAD_OF_MARKETING, BRAND_MANAGER, TRADE_MARKETING_MANAGER, TRADE_MARKETING_OFFICER, MARKETING_EXECUTIVE, MARKETING_ANALYST	TENANT or REGION for field roles
Distribution	DISTRIBUTOR_ADMIN, DISTRIBUTOR_MANAGER, DISTRIBUTOR_STAFF	Own distributor entity and its orders/stock
Analytics	SALES_ANALYST, MARKETING_ANALYST	TENANT, read + export only

Table 4.1 - Seeded role catalog (full permission matrix per role in Appendix B)

4.4 Scope Resolution Rules

- Exactly one active scope node per user (user_scope); changing scope is audited and takes effect on next request, not next login.
- The visible data universe is the subtree of the scope node: a TSO scoped to a territory sees all its routes and outlets without further configuration.
- CSR scope resolves through assignments (outlet_assignment / route_plan), not through the geography ladder alone, because CSRs may share or split beats.
- DISTRIBUTOR_* roles scope to the distributor entity; their visible orders, stock and invoices are those of that distributor only.
- Platform roles never inherit tenant scope. Any tenant peek MUST go through an explicit, time-boxed support grant that is audited end to end.
- Every list, detail, aggregate and export endpoint MUST apply both the tenant filter and the scope filter; aggregates are computed over scoped rows only.

4.5 Worked Example

CSR Rahim, ABC Tobacco

Rahim is user u_rahim, employee e_221, occupying position "CSR - Panchlaish Route 3" in the Sales department. His role CSR grants: outlet.view, visit.create, visit.update, sales.order.create, stock.report, competitor.create. His scope = the two routes he is assigned. He therefore sees exactly 147 outlets, his own visits and his own orders. He cannot see the TSO's territory dashboard aggregates beyond his own contribution, cannot list other CSRs' orders, and cannot export outlet phone numbers outside his routes - the export endpoint applies the identical scope predicate as the screen.

CHAPTER 05

Organization Model

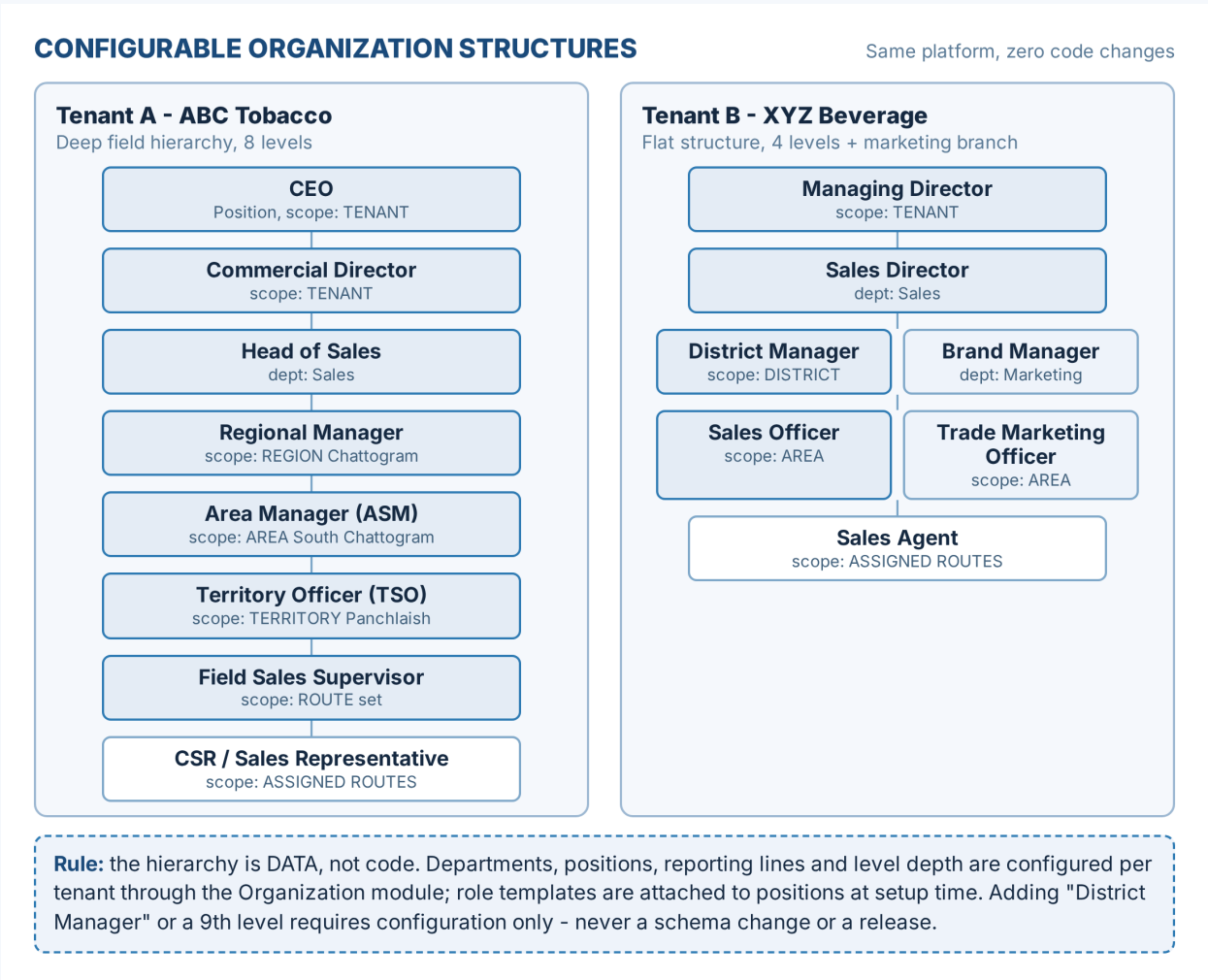
The organization model makes hierarchy configuration, not code. Tenants differ wildly: the reference tobacco tenant runs an eight-level field chain, while a beverage tenant may run four levels with a parallel marketing branch. The platform models organizations as trees of units containing departments and positions; people occupy positions; positions carry role templates and report to other positions. Reports, target cascades and approval routing all consume the same structure, so a tenant re-organization is a data operation with an audit trail, not a development project.

5.1 Structural Entities

Entity	Purpose	Key rules
Organization	Root container for a tenant legal entity or business vertical	One or more per tenant; each has its own unit tree
Business Unit	Optional grouping (e.g. Tobacco Division, Beverage Division)	Nests; used as scope level DIVISION
Department	Functional grouping: Sales, Marketing, Finance, HR, Distribution	Belongs to a unit; positions belong to a department
Position	A seat: "Area Manager, South Chattogram"	Reports to another position (manager_link); carries a default role; single incumbent or vacancy
Employee	A person record (HR identity) independent of login	May exist without a user account; positions held over time tracked via employee_position
User account	Credential + identity used to sign in	1-1 (usually) with employee; disabled accounts do not delete employees

Table 5.1 - Organization module entities

5.2 Two Contrasting Configurations



4. Deactivating a position **MUST** require zero active incumbents or force reassignment; historical references to the position remain valid.
5. Manager resolution for approvals walks position report-to links; if the linked position is vacant, approval escalates to the next occupied ancestor.
6. All organization changes (create/move/rename/deactivate) are audited with before/after snapshots.

5.4 Module Deliverables and Acceptance Criteria

Deliverable	Acceptance criterion
Org tree CRUD	Create/edit/move/rename/deactivate units, departments, positions with validation and audit
Position-employee binding	Assign/release incumbents with effective dates; history queryable
Hierarchy integrity	Cycle detection on save; orphan-position report; vacancy report
Approval chain resolution	Given any position, compute approver chain up to tenant root with escalation on vacancy
Scope attachment UI	Admin attaches scope node to position or user with effect date
Tenant isolation	All org entities tenant-filtered; cross-tenant reference attempts fail with 403/404 and audit entry

Table 5.2 - Organization module acceptance criteria

CHAPTER 06

Master Data I: Employee, User & Geography

Master data modules define the nouns everything else references. They share a common interaction contract: server-side paginated, sortable, filterable lists; create and edit forms with field-level validation; soft delete with reference checks; CSV/Excel export under the same scope filter as the screen; full audit coverage; and bulk import with a dry-run validation report. This chapter specifies employees and users, then the two geography systems; Chapter 7 covers distributors, outlets and products.

6.1 Employee & User Management

Employees are people; users are credentials. The separation lets a tenant onboard its full workforce before granting logins, and lets HR deactivate staff without destroying audit identity. A user account links to at most one employee, and account creation is an audited administrative action with an invitation flow by default.

Field group	Fields
Identity	employee_code (unique/tenant), full_name, photo, gender, date_of_birth, national_id
Contact	phone (unique/tenant), email, present_address, permanent_address

Field group	Fields
Employment	joining_date, status (ACTIVE, INACTIVE, SUSPENDED, TERMINATED, ON_LEAVE), department, position, manager (via position)
Geography	region, area, territory (derived from position scope, editable only via scope change)
Account	linked user_id, roles, last_login_at, mfa_enrolled

Table 6.1 - Employee data dictionary (column-level DDL: employee)

Resource	Create	Read / List	Update	Delete	Export
Employee	HR/T	HR/T/S (scope-limited)	HR/T	Soft delete, HR/T only when no user link	HR/T (scope-limited)
User account	T (invite)	T (never shows passwords)	T (roles, status, reset)	Deactivate only - never hard delete	T
Role assignment	T	T	T (audited, effective next request)	n/a	T
Scope assignment	T	T	T (audited)	n/a	n/a

Table 6.2 - Employee/User CRUD matrix (T=Tenant admin, HR=HR manager role, S=Supervisor within scope)

Business rules

- employee_code and phone uniqueness are per tenant, enforced by unique indexes.
- Status transitions TERMINATED or INACTIVE MUST immediately revoke sessions and tokens; ON_LEAVE keeps sessions but can block order creation via a settings flag.
- Deleting an employee is prohibited when any transactional reference exists; deactivate instead. This is a MUST.
- Every status change, role change and scope change writes an audit entry with old and new values and notifies the affected user.

6.2 Geography: Two Systems, One Purpose

The platform maintains a reference geography (country, division, district, upazila, city - the administrative hierarchy seeded once per platform, shared by all tenants read-only) and a tenant-configurable sales geography (region, area, territory, route). Sales geography nodes may optionally point at reference geography entries for mapping, but nothing depends on that link. Tenants with different depth (e.g. an extra "division" layer) are supported by allowing regions to nest under a business unit and by configuring which levels exist and are exposed per tenant.

Level	Tenant-configurable	Scope attachable	Typical children
Region	Yes	Yes (REGIONAL_MANAGER)	Areas

Level	Tenant-configurable	Scope attachable	Typical children
Area	Yes	Yes (AREA_MANAGER)	Territories
Territory	Yes	Yes (TERRITORY_OFFICER)	Routes, outlets
Route	Yes	Yes (SUPERVISOR/CSR)	Outlets (ordered by route_stop)
Outlet	Yes (entity, not level)	Via assignment	n/a

Table 6.3 - Sales geography levels

Resource	Create	Read / List	Update	Delete	Export
Reference geography	Platform seed only	All (read-only reference)	Platform only	Never	n/a
Region/Area/Territory	TENANT_ADMIN	All roles (scope-filtered)	TENANT_ADMIN	Soft delete when empty only	TENANT_ADMIN, ANALYST
Route	TSO upward	Scope-filtered	TSO/ASM	Soft delete when no plan references	TSO upward

Table 6.4 - Geography CRUD matrix

Acceptance criteria

- Moving a node re-parents its subtree atomically; a node cannot become its own descendant.
- Deleting a geography node with active outlets, plans or targets is rejected with a reference report.
- Scope ladder queries (subtree of node X) return in under 50 ms at reference scale via materialized path or closure table.
- All geography lists support search by name/code and filter by parent.

6.3 API Surface (this module)

Method	Endpoint	Permission	Scope rule
GET	/api/v1/employees	employee.view	Scope filter applied
POST	/api/v1/employees	employee.create	TENANT_ADMIN/HR
GET	/api/v1/employees/{id}	employee.view	Within scope
PATCH	/api/v1/employees/{id}	employee.update	Within scope, HR/T
POST	/api/v1/employees/{id}/status	employee.status	HR/T
GET	/api/v1/users	user.view	TENANT_ADMIN
POST	/api/v1/users/invite	user.create	TENANT_ADMIN
PATCH	/api/v1/users/{id}/roles	user.roles	TENANT_ADMIN
GET	/api/v1/geo/regions ... /routes	geo.view	Scope-filtered subtree

Method	Endpoint	Permission	Scope rule
POST/PATCH/DELETE	/api/v1/geo/{level}	geo.manage	TENANT_ADMIN

Table 6.5 - Representative endpoints (complete catalog in Chapter 20)

CHAPTER 07

Master Data II: Distributor, Outlet & Product

These three modules complete the master data spine. Distributors are business partners with their own portal users; outlets are the retail endpoints where demand originates; products are the hierarchy the whole platform sells. As in Chapter 6, every list is server-side paginated and every export inherits the caller scope. All three modules are feature-flag aware: a tenant that disables Marketing or Inventory simply does not see the related tabs, but the master data itself is always present because sales depends on it.

7.1 Distributor Management

Field group	Fields
Identity	distributor_code (unique/tenant), name, owner_name, photo/logo
Contact	phone, email, address, latitude, longitude
Trade	territory (binding), credit_limit (BDT minor units), opening_balance, payment_terms_days
Status	ACTIVE, INACTIVE, SUSPENDED, TERMINATED
Users	distributor_user rows: admin/manager/staff with portal roles

Table 7.1 - Distributor data dictionary (column-level DDL: distributor)

Business rules

- A distributor is bound to exactly one territory at a time; moving it is an audited operation that reassigns its outlets transitively only via explicit reassignment tooling, never implicitly.
- Credit limit enforcement happens at order approval: approved-order value plus outstanding MUST NOT exceed credit_limit unless an override permission is used (audited).
- Distributor portal users authenticate against the same identity system, but their role family and entity scope confine them to their distributor.
- Deleting a distributor with transactions is prohibited; terminate instead.

7.2 Outlet Management & Segmentation

Field group	Fields
Identity	outlet_code (unique/tenant), name, owner_name, photo
Contact + geo	phone, address, latitude, longitude (captured on creation via device GPS with map correction)
Classification	outlet_type (Retail Shop, Wholesale, Super Shop, Convenience Store, Tea Stall, Other), channel, segment A/B/C/D
Placement	territory, route (via route_plan/assignment), distributor
Status	ACTIVE, INACTIVE, BLACKLISTED, POTENTIAL

Table 7.2 - Outlet data dictionary (column-level DDL: outlet)

Segmentation is tenant-configurable: the platform seeds an A/B/C/D scheme with editable criteria (monthly potential value, shelf count, chilled capacity) and tenants may rename thresholds or add criteria. Segments drive visit frequency suggestions, POSM targeting and reporting cuts, but they never hard-gate ordering.

Resource	Create	Read / List	Update	Delete	Export
Distributor	TENANT_ADMIN	Sales chain + DISTRIBUTOR_* (own entity)	TENANT_ADMIN	Terminate only	TENANT_ADMIN, ANALYST
Distributor user	DIST_ADMIN + TENANT_ADMIN	Own entity only	DIST_ADMIN + TENANT_ADMIN	Deactivate	n/a
Outlet	TSO upward; CSR proposes (POTENTIAL)	Scope-filtered; DIST staff sees own	TSO/ASM; CSR edits limited fields	Soft delete when no visits/orders	Scope-limited
Category/Brand/Product/SKU	TENANT_ADMIN, PRODUCT_MANAGER	All (read)	TENANT_ADMIN, PRODUCT_MANAGER	Soft delete when unreferenced	All (scope-filtered)
Price list	TENANT_ADMIN	All (read)	TENANT_ADMIN with approval option	Never (versioned)	All (read)

Table 7.3 - Distributor/Outlet/Product CRUD matrix

7.3 Product Management

Products follow Category - Brand - Product - SKU. The SKU is the sellable unit with pack size, unit, barcode and prices; orders, stock and targets all reference SKUs. Price lists are versioned effective-dated objects so historical orders keep their original prices, and per-territory price overrides are supported through prioritized price list assignment.

- SKU fields: sku_code (unique/tenant), name, brand_id, pack_size, unit, barcode, mrp, selling_price, cost_price (visible only to permitted roles), tax_rate, status, image.

- MRP/selling_price changes are either immediate or effective-dated per tenant settings; both are audited with old/new values.
- Cost price MUST be excluded from every payload for roles without product.cost.view - including exports and mobile cache.
- Deactivating a SKU blocks new order lines but keeps history and reports intact.
- Bulk import (CSV/XLSX) with dry-run validation, error report per row, and idempotent re-runs is a MUST for products and outlets.

7.4 Module API Surface

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/distributors	distributor.view / distributor.create	List scope-filtered; create TENANT_ADMIN
GET/PATCH	/api/v1/distributors/{id}	distributor.view / distributor.update	Own entity for DIST_*
GET/POST	/api/v1/distributors/{id}/users	distributor.user.manage	DIST_ADMIN, TENANT_ADMIN
GET/POST	/api/v1/outlets	outlet.view / outlet.create	Scope-filtered lists; CSR proposals flagged POTENTIAL
GET/PATCH	/api/v1/outlets/{id}	outlet.view / outlet.update	Within scope
POST	/api/v1/outlets/import	outlet.import	TENANT_ADMIN (dry-run + commit)
GET/POST	/api/v1/products	product.view / product.create	All read; manage restricted
GET/POST	/api/v1/price-lists	product.price.manage	Versioned, effective-dated

Table 7.4 - Representative endpoints for master data II

CHAPTER 08

Field Force: Routes, Visits & CSR Mobile Flow

The field force module is the operational heart of the platform and the CSR mobile experience is its front door. The design principle is speed: a CSR should complete a full outlet visit - check-in, stock, order, competitor note, check-out - in under three minutes on a low-end phone with unstable connectivity. Everything else (management dashboards, coverage analytics) consumes the data this flow produces.

8.1 Route / Beat Management

- A route is an ordered sequence of outlets (route_stop rows) with a planned day pattern: weekly cycle (Mon Route A, Tue Route B ...), fixed interval, or ad-hoc.
- Route plans (route_plan) attach a CSR and a date pattern to a route; conflicts (same CSR, same day, two routes) are validated at save.
- Route optimization is out of scope for v1; stop sequence is manual with drag-ordering, but the data model stores sequence explicitly for future optimization.
- Missed-stop detection compares actual visits against planned stops per day and feeds the TSO/Supervisor dashboards.

8.2 Visit Lifecycle

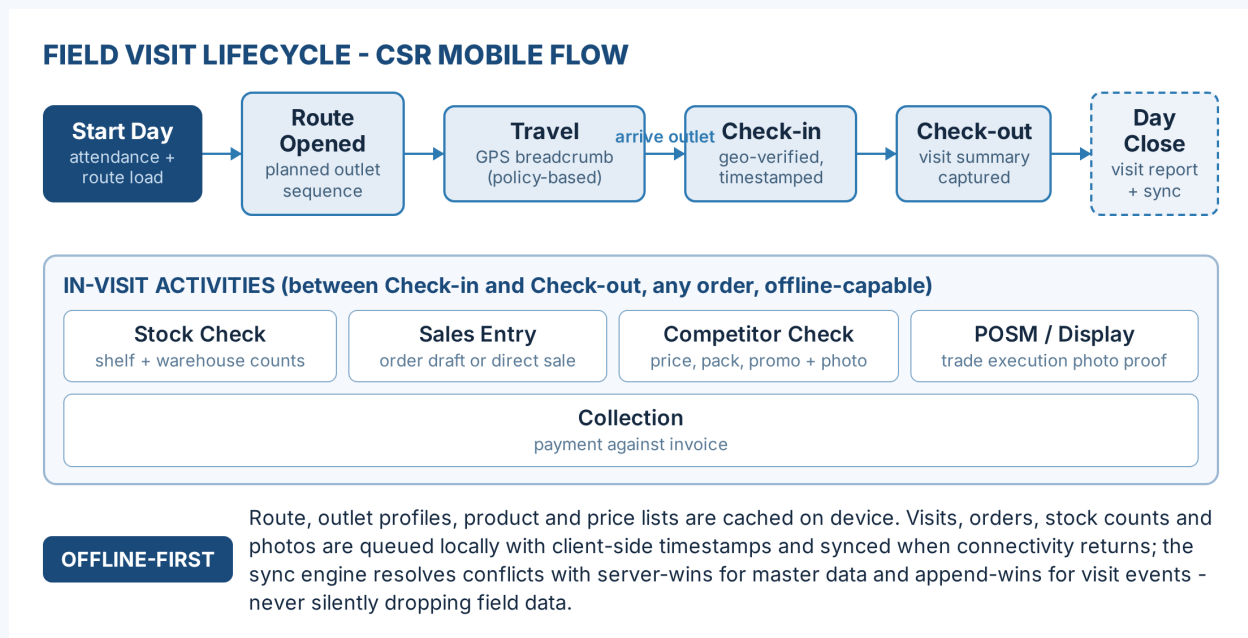


Figure 8.1 - Field visit lifecycle, in-visit activities and the offline sync contract

A visit (visit row) is anchored to an outlet and a planned stop (optional). Check-in captures GPS coordinates and validates distance against the outlet location using a tenant-configurable radius (default 150 m); out-of-radius check-ins require a reason and are flagged, never silently accepted. Activities (visit_activity rows) are typed children: stock checks, order drafts, competitor notes, POSM executions, collections. Check-out writes the visit summary: duration, distance walked, order value, next-visit date.

Resource	Create	Read / List	Update	Delete	Export
Route/plan	TSO/ASM	Scope-filtered	TSO/ASM	Soft delete when unused	TSO upward
Visit	CSR (device)	CSR own; chain upward reads aggregates	CSR until day close; locked after sync	Never (append-only)	TSO upward

Resource	Create	Read / List	Update	Delete	Export
Visit activity	CSR (device)	Within visit	CSR until sync	Never	n/a
Attendance	CSR (device)	Supervisor within scope	Supervisor correction (audited)	Never	Supervisor

Table 8.1 - Field force CRUD matrix

8.3 CSR Mobile Experience Requirements

1. Home shows, in one screen: today target vs achieved, planned outlets, visited so far, orders value today, and a single tap into the current route.
2. Route screen lists today's stops in order with visited/pending status, distance and tap-to-navigate.
3. Outlet screen surfaces last visit summary, outstanding balance, last 3 order lines and the big Check-in button.
4. Order entry works fully offline with cached SKU list and prices; quantity, free item and discount fields validated locally; order drafts queue for sync.
5. All screens usable one-handed, minimum 44 px touch targets, Bengali/English label toggle, and readable in direct sunlight (high-contrast theme support).
6. Sync indicator is always visible; the user never wonders whether data was uploaded.

8.4 Offline Sync & Conflict Resolution (MUST Set)

- Device cache: routes, stops, outlet profiles, SKU catalog, prices, last visit summaries - refreshed opportunistically when online.
- Outbox queue: every offline write is a queued, idempotent event with client UUID, device timestamp and monotonic sequence.
- Conflict policy: master data is server-wins; visit events and activities are append-wins (never dropped); order drafts conflict-resolve by re-validating prices and stock on ingest and flagging deltas to the CSR.
- Photo uploads retry with exponential backoff; attachment rows are created on ingest, not on capture, so queue loss never orphans references.
- The sync endpoint MUST be rate-limited per device and MUST accept batch envelopes of up to 200 events.

CHAPTER 09

Sales Operations: Orders, Transactions & Performance

Sales converts field activity into revenue records. The sales order is a workflow object with a strict state machine; the sales transaction is the immutable financial fact derived from it. Every transaction row denormalizes the full organizational and geographic dimension set (CSR, outlet, route, territory, area, region, distributor, brand, SKU) at write time, which makes every dashboard in Chapter 14 a simple indexed aggregation rather than a multi-join nightmare.

9.1 Sales Order Lifecycle

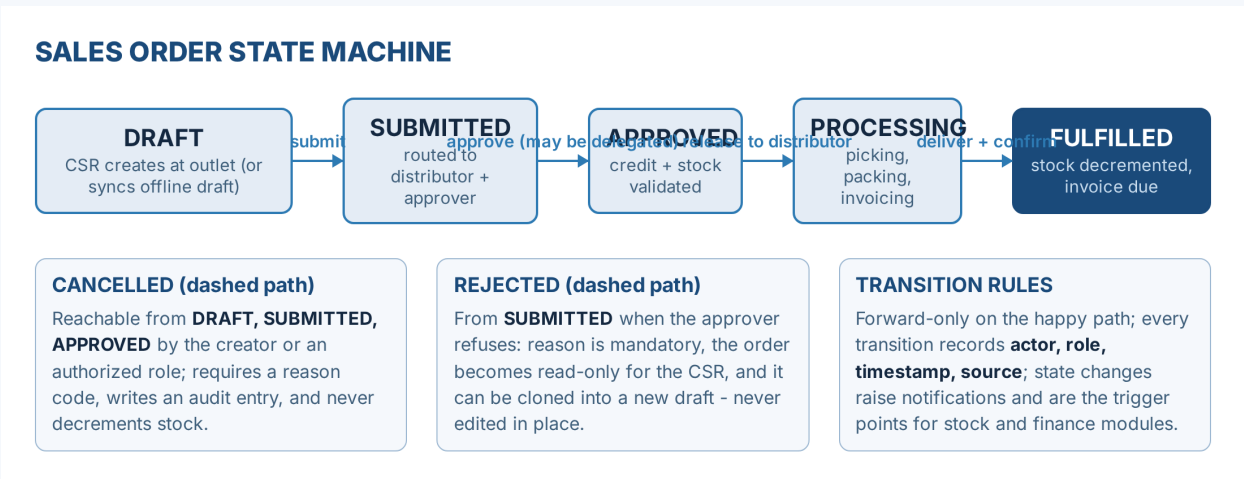


Figure 9.1 - Sales order state machine with cancel/reject paths and transition rules

Field group	Fields
Header	order_number (per-tenant sequence), date, csr_id, outlet_id, distributor_id, status, source (MOBILE/WEB/SYNC), currency
Lines	sku_id, qty, free_qty, unit_price (snapshot), discount_amount, line_total
Totals	gross_total, discount_total, tax_total, net_total (all BDT minor units, integers)
Workflow	approved_by, approved_at, rejection_reason, cancel_reason, fulfilment refs

Table 9.1 - Sales order data dictionary (column-level DDL: sales_order)

Business rules (MUST set)

- Line unit_price snapshots the price list at creation; later price edits never touch existing orders.
- Quantity must be a positive multiple of the SKU's order multiple when one is configured; free quantity is tracked separately for scheme reporting.

- Approval follows the Approval Engine (Chapter 12.4): auto-approve below a tenant-configurable value, route above it; credit limit check runs before APPROVED is reachable.
- Transition to PROCESSING and FULFILLED is performed by distributor portal users; every transition writes order_status_history with actor and timestamp.
- Cancellation is only possible before PROCESSING and requires a reason code; fulfilled orders are corrected by returns, not edits.
- An order created offline replays through the identical validation path on ingest; if validation fails, it surfaces as a flagged draft, never a silent drop.

9.2 Sales Transactions

On FULFILLED, the platform writes one sales_transaction row per order line plus a header fact row, capturing quantity, gross, discount, net and the entire dimension spine. Transactions are immutable; corrections happen through new rows (returns, adjustments) flagged with reference to the originating row. All performance reporting reads transactions, never live orders, except for the in-progress day views.

9.3 Performance Metrics (definitions are contractual)

Metric	Formula	Example (BDT)
Achievement %	actual / target x 100	850,000 / 1,000,000 = 85%
Variance	actual - target	850,000 - 1,000,000 = -150,000
Growth %	(actual - prev_period) / prev_period x 100	(850k - 720k) / 720k = 18.1%
Strike rate	productive_calls / visited_outlets x 100	31 / 42 = 73.8%
Coverage	visited_outlets / planned_outlets x 100	42 / 50 = 84%
Avg order value	net_sales / orders	850,000 / 64 = 13,281

Table 9.2 - Metric definitions used verbatim across dashboards and reports

85% Achievement example	-150,000 Variance (BDT)	73.8% Strike rate	18.1% Growth vs prev
-----------------------------------	-----------------------------------	-----------------------------	--------------------------------

9.4 CRUD Matrix & API Surface

Resource	Create	Read	Update	Delete	Export
Sales order	CSR (mobile/web), Sales admin	Creator; chain upward; DIST (own)	DRAFT only by creator; else workflow	DRAFT only; else cancel	Chain upward
Transaction	System on fulfilment	Scope-filtered; ANALYST read	Never (immutable)	Never	ANALYST, chain upward

Resource	Create	Read	Update	Delete	Export
Status history	System	With order	Never	Never	n/a

Table 9.3 - Sales CRUD matrix

Method	Endpoint	Permission	Scope rule
POST	/api/v1/sales/orders	sales.order.create	CSR own outlets; web admin within scope
GET	/api/v1/sales/orders	sales.order.view	Scope-filtered
POST	/api/v1/sales/orders/{id}/submit	sales.order.submit	Creator or admin
POST	/api/v1/sales/orders/{id}/approve reject	sales.order.approve	Approver per workflow
POST	/api/v1/sales/orders/{id}/cancel	sales.order.cancel	Pre-processing + reason
GET	/api/v1/sales/transactions	sales.transaction.view	Scope-filtered, server-side agg

Table 9.4 - Sales API surface (representative)

CHAPTER 10

Target Management & Allocation Cascade

Targets are the platform's goal machinery. A target definition states WHAT is measured (value, quantity, coverage, collection, visits, new outlets), for WHICH period (daily to yearly) and at WHICH scope node. Allocations cascades the number downward with strict traceability; achievement computes upward automatically from transactions. The cascade is the conceptual bridge between the organization tree, the sales geography and the reporting system, and it is validated so that children always sum to parents.

10.1 Target Types & Periods

Type	Unit	Measured from	Typical levels
Sales value	BDT minor units	sales_transaction.net	Company to CSR
Sales quantity	CS (or SKU units)	sales_transaction.qty	Company to CSR
Product quantity	SKU units	transactions filtered by SKU	SKU campaigns
Outlet coverage	% or count	visits vs planned	Territory to CSR
New outlet	count	outlets created + activated	Territory to CSR
Collection	BDT minor units	payment rows	Territory to CSR
Visit count	count	visit rows	CSR, Supervisor

Table 10.1 - Target type catalog (extensible per tenant)

10.2 The Allocation Cascade

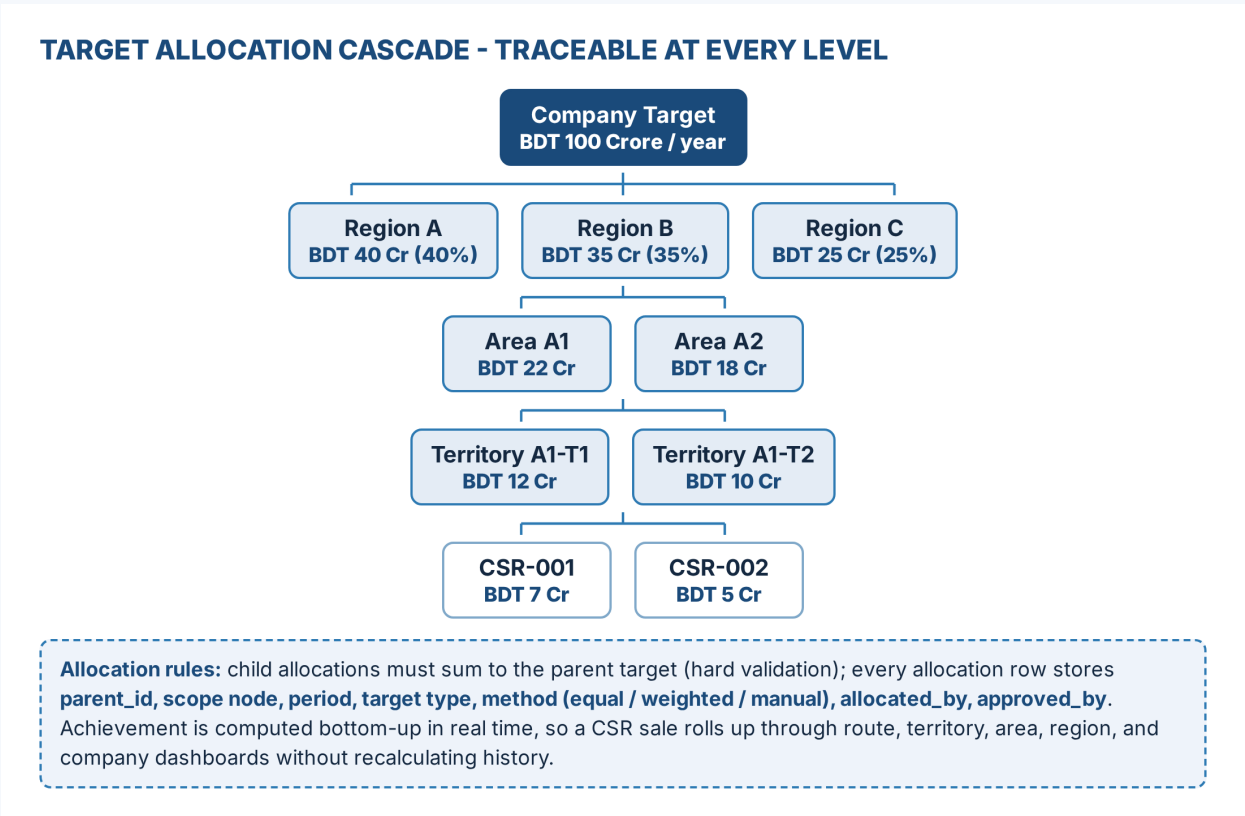


Figure 10.1 - Target allocation cascade with the BDT 100 crore worked example

Allocation methods: EQUAL (split evenly among children), WEIGHTED (by a chosen weight such as last-period achievement or outlet count), and MANUAL (admin enters numbers; platform validates the sum). An allocation row stores **parent_id, scope node, period, method, allocated_by** and **approved_by**, giving end-to-end traceability: any CSR's yearly number can be explained as a chain of allocations from the company number. Re-allocation mid-period is allowed, versioned, and always audited.

10.3 Achievement Computation

- Achievement is computed bottom-up in real time from transactions for value/quantity types, and from visit/coverage facts for activity types.
- Dashboards read pre-aggregated daily rollups (**achievement_snapshot**) that the scheduler rebuilds every N minutes; live day views may query raw rows.
- Every achievement number **MUST** be reproducible: given scope node + period, the engine returns target, actual, achievement %, variance and prior-period comparison from the same code path.
- Period boundaries follow tenant fiscal calendar settings (Chapter 41 of settings; fiscal year start month configurable).

10.4 CRUD Matrix & API Surface

Resource	Create	Read	Update	Delete	Export
Target definition	HEAD_OF_SALES, TENANT_ADMIN	All (own scope rows)	Before approval	Draft only	Chain upward
Allocation	Owner of parent node	Subtree	Versioned re-allocation	Never (version)	Chain upward
Approval	Workflow engine	Approvers	n/a	n/a	n/a

Table 10.2 - Target CRUD matrix

Method	Endpoint	Permission	Scope rule
POST	/api/v1/targets	target.create	Own node only
POST	/api/v1/targets/{id}/allocate	target.allocate	Parent owner; sum validated
POST	/api/v1/targets/{id}/submit	target.submit	Owner
POST	/api/v1/targets/{id}/approve	target.approve	Approver per workflow
GET	/api/v1/targets/achievement	target.achievement.view	Scope-filtered rollout query
GET	/api/v1/targets/traceability/{allocationId}	target.achievement.view	Returns allocation chain

Table 10.3 - Target API surface

CHAPTER 11

Marketing: Campaigns, Trade Marketing & Competitors

Marketing in a route-to-market platform is execution-first: campaigns translate into outlet-level tasks that CSRs and trade marketing officers perform on their beats, and the platform measures what actually happened in the field rather than what was planned in a slide. The marketing suite therefore spans campaign planning, trade execution tracking with photo proof, POSM asset management and a competitor intelligence stream captured directly by the field force.

11.1 Campaign Management

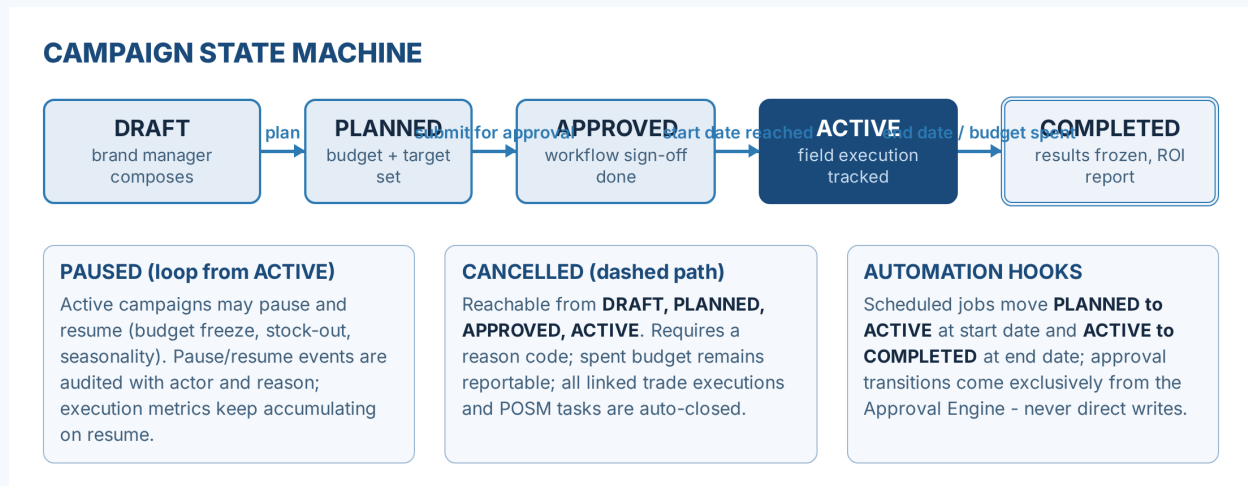


Figure 11.1 - Campaign state machine with pause, cancel and automation hooks

Field group	Fields
Definition	name, code, brand_id, product/sku scope, objective (VISIBILITY, VOLUME, ACTIVATION, LAUNCH)
Window & budget	start_date, end_date, budget (BDT), expected_outlets, region/area/territory targeting
Execution spec	attached POSM kits, display standards, per-visit checklist template
Status	DRAFT, PLANNED, APPROVED, ACTIVE, PAUSED, COMPLETED, CANCELLED

Table 11.1 - Campaign data dictionary (column-level DDL: campaign)

Business rules

- Campaign approval runs through the Approval Engine; budget above a tenant threshold escalates to COMMERCIAL_DIRECTOR or CEO.
- Targeted outlets are materialized at approval time (campaign_execution rows per outlet) so achievement is stable even if routes change mid-flight.
- Execution progress: executed / target outlets, e.g. 430 / 500 = 86%; the number updates from visit activities linked to the campaign.
- Budget consumption (POSM cost, activation fees) is recorded against the campaign and blocks transition to ACTIVE if it exceeds budget before start.

11.2 Trade Marketing & POSM Execution

A trade execution task binds an outlet, a campaign, a display standard and a POSM asset list. The field user executes during a visit, captures photo proof, and the platform scores execution quality (photo present, checklist complete, in-window). Trade dashboards report execution achievement per territory, per brand and per promoter, with drill-down to individual outlet photos for verification. POSM assets are tracked as inventory items with distribution records so the platform can answer "where did the 2,000 danglers go and what did they produce".

11.3 Competitor Intelligence

Captured at visit	Purpose
Competitor brand + product + pack size	Map competitor SKU presence per outlet/territory
Price observed (BDT)	Price war and margin-erosion detection
Availability (in stock / out of stock)	Share-of-shelf estimation over time
Promotion observed + photo	Tactical response planning
Free-text notes	Context for brand managers

Table 11.2 - Competitor record capture template

Competitor records are appended from the CSR mobile flow (one tap from the visit screen) and from trade marketing officers. Reporting aggregates them by territory, brand and time so management sees a competitive heat map without any manual collation. Records are immutable; corrections are new records referencing the original.

11.4 CRUD Matrix & API Surface

Resource	Create	Read	Update	Delete	Export
Campaign	BRAND_MANAGER	Chain + field (scoped)	Before ACTIVE; budget freeze audited	Draft/cancel only	Marketing chain
Campaign execution	System (materialized)	Field (scoped)	Execution status by field user	Never	Marketing chain
POSM asset	MARKETING_EXEC	Marketing chain	MARKETING_EXEC	Deactivate	Marketing chain
Competitor record	CSR/TMO from visit	Chain upward, BRAND_MANAGER	Never (append-only)	Never	Marketing, ANALYST

Table 11.3 - Marketing CRUD matrix

Method	Endpoint	Permission	Scope rule
POST	/api/v1/campaigns	campaign.create	Brand manager upward
POST	/api/v1/campaigns/{id}/submit approve	campaign.approve	Workflow approvers
GET	/api/v1/campaigns/{id}/execution	campaign.execution.view	Scoped execution progress
POST	/api/v1/visits/{id}/trade-execution	trade.execute	CSR on own visit
POST	/api/v1/visits/{id}/competitor	competitor.create	CSR on own visit
GET	/api/v1/marketing/competitor-summary	competitor.view	Aggregated, scoped

Table 11.4 - Marketing API surface (representative)

CHAPTER 12

Inventory, Collection & Approval Engine

Three operational services close the loop between field activity and money and goods. Inventory visibility answers "what stock exists where" without pretending to be a warehouse ERP. Collection management turns fulfilled orders into invoices, payments and outstanding positions. The approval engine is a single configurable workflow service consumed by orders, targets, campaigns, prices, outlets and users, so approval logic is configured once and behaves identically everywhere.

12.1 Inventory Visibility

Stock space	Source	Granularity	Notes
Distributor stock	Distributor portal entries + movement log	SKU x distributor	Opening, received, sold, closing
Outlet stock	CSR stock-check activities	SKU x outlet (spot)	Spot counts, not books; dated by visit
CSR-reported	Stock check + display counts	SKU x visit	Feeds availability metrics

Table 12.1 - Three stock visibility spaces

- stock_movement is append-only: every quantity change is a movement row with type (RECEIVED, SOLD, ADJUST, TRANSFER, RETURN), reference and actor; the two stock tables are projections of it and are rebuildable.
- Negative-closing validation is a hard rule: projections MUST NOT go negative; attempts surface as flagged adjustments for supervisor review.
- Availability metric = outlets where a SKU was observed in stock / outlets visited in period; it feeds the TSO and CEO dashboards.
- v1 explicitly excludes batch/lot, expiry and warehouse bin management; the movement table carries optional batch_ref columns so a future ERP-grade module can extend without migration shock.

12.2 Collection Management

Concept	Rule
Invoice	Raised on FULFILLED per order to the distributor account; line-level from order lines with tax breakdown
Payment	Cash/cheque/digital received by CSR in field or at distributor; allocated to invoices FIFO or manually
Outstanding	invoice_amount - allocated payments; aged buckets 0-30/31-60/61-90/90+ days
Credit exposure	Outstanding + open approved orders vs distributor credit_limit; blocks approval when exceeded

Concept	Rule
Overdue flow	Overdue invoice notifies distributor admin and TSO; configurable block on new orders after N days

Table 12.2 - Collection domain rules

Resource	Create	Read	Update	Delete	Export
Stock movement	System/portal entries	Scoped reads	Never	Never	ANALYST, chain
Distributor stock	DIST staff	Chain + DIST own	Via movements only	Never	Chain
Outlet stock	CSR activities	Chain	Never (spot snapshots)	Never	Chain
Invoice	System on fulfilment	Chain + DIST own + FINANCE	Never (credit notes instead)	Never	FINANCE, ANALYST
Payment	CSR/DIST/FINANCE	Own + chain	Allocation edits audited	Void, never delete	FINANCE, ANALYST

Table 12.3 - Inventory & finance CRUD matrix

12.3 API Surface

Method	Endpoint	Permission	Scope rule
POST	/api/v1/inventory/stock-movements	stock.movement.create	DIST own / FINANCE
GET	/api/v1/inventory/distributor-stock	stock.view	Scoped
GET	/api/v1/inventory/availability	stock.view	Aggregated, scoped
GET	/api/v1/finance/invoices	invoice.view	Own + chain
POST	/api/v1/finance/payments	payment.create	CSR field collection / DIST / FINANCE
GET	/api/v1/finance/outstanding	invoice.view	Aged buckets, scoped

Table 12.4 - Inventory & finance API surface

12.4 Configurable Approval Workflow Engine

Approvals are configured, not coded. An approval workflow declares: the entity type it governs, eligibility conditions (e.g. order value above 25,000 BDT), an approver chain resolved from organization positions, escalation timeouts, and delegation rules. Requests are immutable records of who approved what and when; the engine exposes a clean API that domain modules call on submission and await on decisions.

Entity	Default chain (example)	Escalation
Sales order	Auto below threshold; TSO above; ASM above higher threshold	12h to next position
Target allocation	Head of Sales; CEO above 10 Cr	48h

Entity	Default chain (example)	Escalation
Campaign	Head of Marketing; Commercial Director on budget	48h
Price change	TENANT_ADMIN; COMMERCIAL_DIRECTOR above 5%	24h
Outlet creation (CSR proposal)	TSO then ASM	24h
User / role change	TENANT_ADMIN	24h

Table 12.5 - Seeded workflow configurations (tenant-editable)

CHAPTER 13

Platform Services: Notifications, Files & Audit

Three horizontal services support every module. Notifications fan business events out to humans through swappable channels. File management stores and serves every binary safely behind authorization. Audit logging records every meaningful action in a tamper-evident trail that satisfies both internal governance and external review. All three are feature-flag aware and expose their operations through the same /api/v1 gateway as business modules.

13.1 Notification System

Domain modules publish events; the Notifications module maps each event type to recipient resolution rules (role + scope or explicit user list) and to channels. Provider implementations are behind interfaces - SMTP and a Bangladesh-friendly SMS gateway in v1, push via FCM for the PWA - so providers can be changed per deployment without touching business code. Every notification is stored in-app first (notification table), then dispatched; channel failures retry with backoff and never lose the in-app copy.

Event	Recipients	Channels
Target assigned / approved / missed	Target owner + chain	In-app, push
Order created / approved / rejected	CSR, distributor admin, approver chain	In-app, push, SMS optional
Approval required	Current approver (with escalation)	In-app, push, email
Campaign approved / started / completed	Marketing chain	In-app, email
Outlet created / flagged	TSO, ASM	In-app
Credit limit breach	FINANCE, TSO	In-app, email
User account actions	Affected user + TENANT_ADMIN	In-app, email

Table 13.1 - Core notification event catalog (extensible)

Method	Endpoint	Permission
GET	/api/v1/notifications?unread=true	authenticated
POST	/api/v1/notifications/{id}/read	owner

Method	Endpoint	Permission
POST	/api/v1/notifications/read-all	owner
GET/PATCH	/api/v1/notification-preferences	owner (channel opt-in per event family)

Table 13.2 - Notification API surface

13.2 File & Attachment Management

- attachment rows are polymorphic (entity_type, entity_id) and carry: file id, tenant, entity refs, storage key, mime type, size, sha256, uploaded_by, created_at, scan_status.
- Storage keys are tenant-prefixed and opaque; raw paths NEVER appear in any API response. Downloads go through signed, short-lived URLs re-authorized against the caller scope on every request.
- Upload limits (default 10 MB) and MIME allow-lists (jpeg/png/webp/pdf) are tenant-configurable; images may be auto-downscaled for mobile cache.
- Deletion of attachments follows the owning entity lifecycle; orphan sweep jobs clean unreferenced blobs weekly.

13.3 Audit Logging

Audit coverage is a first-class acceptance criterion, not an afterthought. The audit_log row stores: user, tenant, action, module, entity type and id, old and new value snapshots (JSONB), request ip, user agent and timestamp. Writes are append-only via a dedicated write path; the application database user MUST NOT have UPDATE/DELETE on the audit table (database-enforced tamper evidence).

Must be audited	Must be audited (cont.)
Auth events: login, failed login, lockout, reset, MFA changes	Export execution: report id, filters, row counts
Any create/update/delete on master data	Scope or role changes (with old/new scope)
Order submit/approve/reject/cancel	Tenant administration: provisioning, plan, feature flags
Target allocate / re-allocate / approve	Approval decisions and delegations
Price changes (old/new value mandatory)	Support grants by platform roles (start/end)
Distributor/portal user actions	Attachment upload/download of sensitive classes

Table 13.3 - Mandatory audit coverage (non-exhaustive)

Design guardrail

If an action changes what a user sees, owns or gets paid on, it must be auditable. When in doubt, audit. Audit failures MUST degrade gracefully (queued write) without blocking the business transaction, and MUST alert operators if the queue grows.

CHAPTER 14

Dashboard Specifications (All Roles)

Every role opens the platform onto a purpose-built dashboard whose widgets are computed from the same metric definitions as Chapter 9.3 - one formula, everywhere. Dashboards are scope-aware by construction: a Regional Manager sees the region ladder, a TSO the territory ladder, a CSR the day ladder. Each dashboard below lists its KPI cards, primary widgets, standard filters and drill-down path. All dashboards MUST render within 2 seconds at reference scale by reading the pre-aggregated rollout tables.

14.1 Super Admin (Platform Plane)

Widget	Content	Drill-down
KPI row	Total tenants, active tenants, total users, 30-day active users, MRR proxy	Tenant list
Tenant usage table	Per-tenant: users, orders 30d, storage, plan, status	Tenant detail
System health	API p95 latency, error rate, queue depth, backup freshness	Ops dashboard (external)
Audit spotlight	Recent platform-level admin actions and support grants	Audit viewer

Table 14.1 - Super Admin dashboard

14.2 CEO

Widget	Content
KPI row	MTD revenue (BDT), YTD revenue, achievement %, YoY/MoM growth, active outlets
Region performance	Bar ranking of regions by achievement with target lines
Product performance	Brand/SKU revenue mix donut + top 10 SKU table
Trend	12-month revenue and achievement dual-axis line
Top lists	Top territories, top CSRs, top distributors by contribution

Table 14.2 - CEO dashboard widgets (filters: period, region, brand)

14.3 Head of Sales

- National target ladder: company - region - area - territory with achievement bars and variance flags.
- CSR productivity board: strike rate, coverage, AOV, lines per call, ranked with tenant-configurable weights.
- Distributor health: order fill rate, outstanding aging, credit utilization.

- Gap analyzer: which territory x SKU cells are below par this month with last-period comparison.

14.4 Regional Manager / ASM / TSO (ladder pattern)

The three management levels share one dashboard pattern parameterized by scope: achievement header (target vs actual vs variance), child-node ranking table (areas for RM, territories for ASM, CSRs for TSO), outlet coverage and availability tiles, distributor health tile, and a pending approvals queue. The TSO variant adds CSR day-level activity (visits started/in-progress/completed) and a missed-visit alert list. All three drill from row to the child dashboard in one click, preserving the period context.

14.5 Supervisor

Widget	Content
Team strip	CSR cards: present/absent, current route, visits done, orders today
Visit funnel	Planned - visited - productive with conversion rates
Order board	Orders today by status with stuck-order flags
Exception list	Missed visits, out-of-radius check-ins, failed syncs

Table 14.3 - Supervisor dashboard

14.6 CSR Mobile Dashboard

The CSR home is a single screen, thumb-optimized: Today target vs achieved (value + quantity), planned vs visited vs productive outlets, orders and collection today, sync status chip, and the current route card with the next outlet one tap away. Achievement updates live from synced orders; while offline it shows queued-order projections marked as pending. No management chrome, no nested menus - the design system in Chapter 16 dedicates a mobile layout family to this screen.

14.7 Marketing Dashboard

- Campaign portfolio: budget burn, execution %, window countdown per campaign.
- Trade execution heat: executed/target outlets by territory with photo drill-down.
- Brand and SKU performance tiles reusing sales metrics with marketing UTA attribution (campaign-linked orders).
- Competitor activity: record counts and price observations by territory and brand over time.
- POSM distribution: assets issued vs execution proof count.

14.8 Cross-Cutting Dashboard Rules

1. Every widget states its scope and period in place - no orphan numbers.
2. Every KPI card is a hyperlink to the report or list behind it with filters pre-applied.
3. Empty, loading and error states are designed per widget (Chapter 16 components), never an unstyled spinner over a blank card.
4. Real-time columns (day ladders) refresh on sync events; period ladders refresh on rollup completion with a subtle updated-at stamp.
5. Dark and light themes must render identical information hierarchy (Chapter 16 tokens).

CHAPTER 15

Reporting Engine & Report Catalog

Reports are configured, not hand-coded one by one. The report engine holds parameterized report definitions - dataset, dimensions, measures, filters, groupings, rendering hint - and executes them against the read replica with mandatory row limits and scope enforcement. Hard-coded reports are allowed only where custom layout demands it, and they **MUST** reuse the same metric definitions. All exports run as background jobs producing Excel/CSV/PDF artifacts in object storage with signed download links.

15.1 Engine Requirements

- Definition schema: dataset, dimensions[], measures[], filters[], period control, grouping level, sort, visualization (table/pivot/bar/line), row limit, export formats.
- Execution: read-replica routing, statement timeout, hard row cap (default 100k, tenant-configurable), and the caller scope injected as a non-removable filter.
- Every generated artifact is registered in attachment storage and audited (who exported what with which filters and row counts).
- Scheduled reports: cron per tenant, delivery by email with signed links, per-tenant retention policy (default 90 days).

15.2 Report Catalog (v1 **MUST** ship all of these)

Report	Dimensions	Measures	Default filters
Sales summary	Period, region/area/territory/CSR	Net value, qty, orders	Period, geo, brand
Product sales	Brand, SKU, pack	Qty, value, achievement	Period, geo
Brand performance	Brand	Value, share %, growth	Period, geo
Geography sales	Region, area, territory	Value, achievement %	Period

Report	Dimensions	Measures	Default filters
CSR performance	CSR	Value, strike rate, coverage, AOV	Period, territory
Distributor sales	Distributor	Primary/secondary value, fill rate	Period
Outlet sales	Outlet, segment, type	Value, orders, last visit	Territory, period
Target vs achievement	Any scope node	Target, actual, %, variance	Period, geo
Growth report	Geo x period comparison	Value, growth %	Periods, geo
Campaign report	Campaign, territory	Executed/target outlets, spend, uplift	Campaign, period
Trade marketing execution	Campaign, brand, territory	Executed %, photo proof rate	Campaign
Collection report	Distributor, CSR, invoice	Collected, outstanding, aging	Period, distributor
Stock report	Distributor/outlet, SKU	Closing qty, days of cover	Date, geo
Competitor report	Competitor brand, territory	Records, avg price, presence %	Period, geo
Visit log	CSR, outlet, date	Visits, duration, activities	Period, route
Audit report	User, module, action	Event count + detail view	Period, module

Table 15.1 - v1 report catalog (16 reports)

Method	Endpoint	Permission
GET	/api/v1/reports	reports.view (definitions visible to role)
POST	/api/v1/reports/{key}/run	reports.view + module view perm of dataset
POST	/api/v1/reports/{key}/export	reports.export
GET	/api/v1/reports/jobs/{jobId}	reports.view (own job)
CRUD	/api/v1/reports/schedules	reports.schedule

Table 15.2 - Reporting API surface

CHAPTER 16

UI/UX Design System: Premium Dual-Theme

The product ships with TWO first-class themes - a premium LIGHT theme and a deep-premium DARK theme - switchable by an always-visible toggle in the top bar, persisted per user, with system-preference detection as the default. Both themes are derived from ONE token set: the same hue family, the same typography, the same spacing; only the surface luminosity scales invert. The corporate blue family (deep navy #0a1628 night tones for dark, blue-white #f5f8fc daylight tones for light) is the brand baseline, and the accent ladder sits between them.

16.1 Theme Contract (MUST Set)

1. A single toggle switches light/dark instantly with no reload; choice persists per user across devices; first visit follows OS preference.
2. Every component defines both themes via tokens - hard-coded colors in components are forbidden and MUST fail code review.
3. Contrast: body text at least 4.5:1, large text and UI glyphs at least 3:1 against their surface, in BOTH themes (WCAG 2.1 AA).
4. Charts, maps, status chips and photos define theme-aware treatments (dark-mode chart grids, dimmed imagery) so no widget looks pasted-in.
5. Print and export styles always render in light theme regardless of toggle.

16.2 Design Tokens

Token family	Light values (premium daylight)	Dark values (deep navy)
bg/page	#f5f8fc (blue-white)	#0a1628 (night navy)
bg/surface	#ffffff cards on #f5f8fc	#11203a cards on #0a1628
bg/sunken	#e4ecf5	#0d1a30
text/primary	#142840	#e8f0f8
text/muted	#5a7a96	#7a9bb8
accent/primary	#1a4a7a	#4da8da
accent/glow	#2d7ab3	#2d7ab3
status/success-warn-danger	#2e7d5b / #b0782d / #b04a3a (muted, both themes adjusted for contrast)	same hues, luminosity raised
border/subtle	#c0d0e2	#233a5c

Table 16.1 - Core color tokens (single hue family ~215 degrees, two luminosity worlds)

Token family	Values
Type scale	Display 40 / H1 28 / H2 20 / H3 16 / Body 14 / Caption 12 (desktop px); mobile body 16 min
Font family	Inter (Latin) with tabular numerals for KPIs; Bengali companion font for bn locale
Spacing	4 px base grid: 4, 8, 12, 16, 24, 32, 48; cards 16-24 inner padding
Radius	Inputs and buttons 8, cards 12, chips 999
Elevation	Light: soft shadows; Dark: luminous 1px borders + glow accents instead of shadows
Density	Comfortable (management) and compact (data tables) mode, tenant-switchable

Table 16.2 - Typography, spacing and shape tokens

16.3 Component Library Inventory

Family	Components
Navigation	Top bar with theme toggle + notifications, sidebar (desktop), bottom tab bar (CSR mobile), breadcrumbs
Data display	KPI card, data table (server-driven), pivot, donut/bar/line charts, heat grid, map view, timeline, photo gallery
Input	Text/number/date, select with search, cascader (geo ladder), tag picker, quantity stepper, GPS capture, signature, file dropzone
Feedback	Empty state, skeleton loaders, error state with retry, toast, banner, badge, offline chip
Workflow	Approval card, status stepper, audit drawer, comment thread
Layout	Dashboard grid, split view, drawer, modal, wizard, print sheet

Table 16.3 - Component inventory (all components ship in both themes)

16.4 Responsive & Device Rules

- Breakpoints: 360 (small phone), 768 (tablet), 1024 (small laptop), 1440+ (management desktop). Management screens target 1280 first, tablet second.
- CSR mobile screens are 360-430 px first, one-handed reach, bottom-tab navigation, 44 px minimum targets, and remain fully functional offline.
- Data tables switch to card-list layout under 768 px with the three most important columns preserved and the rest behind a detail sheet.
- Charts downgrade gracefully: map view requires connectivity, all other widgets degrade to cached last-sync data with a clear timestamp.

16.5 Accessibility & Localization

- WCAG 2.1 AA: contrast, focus rings, keyboard operation for all management flows, screen-reader labels on KPI cards and charts.
- Locale: English + Bengali (bn) label catalogs from day one; currency and number formatting via a single formatting service (lakh/crore grouping for BDT is the default).
- Date/time display follows tenant timezone with per-user override; fiscal calendar label support (month names per tenant setting).

CHAPTER 17

Frontend Page & Screen Inventory

This chapter enumerates every screen with its route, primary permission and the CRUD operations it exposes. It is the wiring diagram between the module specs and the design system. Route paths are indicative but the permission column is contractual: a screen **MUST NOT** render data the caller could not have fetched through the API.

Route	Screen	Permission	Operations
/login, /forgot-password	Auth screens	public	login, reset request
/dashboard	Role dashboard (Ch 14)	dashboard.view	read, drill-down
/employees /users /roles	People & access admin	employee.*, user.*	CRUD, status, roles, scope
/organization	Org tree builder	org.manage	CRUD units/departments/positions
/geo	Geography ladders	geo.view / geo.manage	CRUD nodes, subtree view
/distributors	Distributor list/detail	distributor.*	CRUD, users, credit, stock tab
/outlets	Outlet list/detail/map	outlet.*	CRUD, assign, segment, import
/products	Catalog (cat/brand/product/SKU)	product.*	CRUD, price lists, import
/routes	Route & plan builder	route.*	CRUD routes, plans, stops
/visits	Visit log + detail	visit.view	read, export, flags
/sales/orders	Order list/detail	sales.order.*	create, submit, approve, cancel
/sales/transactions	Transaction explorer	sales.transaction.view	read, export
/targets	Target console + cascade	target.*	create, allocate, approve, trace
/campaigns	Campaign list/detail	campaign.*	CRUD, execute tracker
/trade	Trade execution board	trade.execute / view	review, photo verify
/competitors	Competitor intel	competitor.view	read, export
/inventory	Stock visibility	stock.view	read, movements, availability
/finance	Invoices/payments/outstanding	invoice.*, payment.*	read, pay, allocate, age
/approvals	My approvals queue	workflow.decide	approve, reject, delegate
/reports	Report gallery + runner	reports.*	run, export, schedule
/notifications	Notification center	authenticated	read, preferences
/audit	Audit viewer	audit.view	search, filter, export
/settings	Tenant settings	tenant.settings	profile, fiscal, approvals, flags
/platform/tenants	Super admin tenants	platform.tenant.*	provision, suspend, plan, flags
/csr	CSR mobile shell (PWA)	CSR role	route, visit, order, sync

Table 17.1 - Complete screen inventory with permission wiring

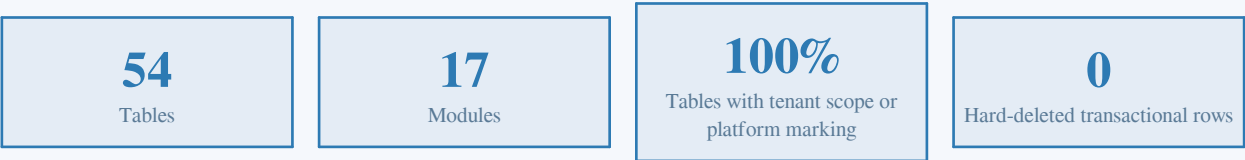
Screen-level acceptance rule

Every screen must implement the five states (loading, empty, error, no-permission, offline) and every list must be server-paginated. A screen that renders client-filtered full datasets is a defect even if it looks fast on demo data.

CHAPTER 18

Database Design Standards & ERD

The database is PostgreSQL-flavored in its DDL (Chapter 19) but the rules are portable to any mature relational engine. Fifty-four tables implement the seventeen modules; every tenant-owned table carries the standard column block; every foreign key is declared; every list query path is indexed. This chapter fixes the conventions and shows the domain-level ERD; the DDL chapter is the column-level contract.



18.1 Naming & Typing Conventions

Convention	Rule
Identifiers	snake_case; tables plural; join tables named a_b; index ix_table_cols; fk fk_table_ref; uq_table_cols
Primary keys	uuid v4 (gen_random_uuid()) - safe for offline client generation and merge-free sync
Tenant column	tenant_id uuid NOT NULL on every tenant-owned table; platform tables document their plane explicitly
Money	bigint minor units (poisha); never float; currency column alongside when multi-currency is possible
Time	timestamptz everywhere, UTC storage, tenant timezone at render
Enums	CHECK constraints or enum types with migration-managed value lists; status vocabularies fixed by this spec
Soft delete	deleted_at timestamptz NULL; unique indexes are partial (WHERE deleted_at IS NULL); transactional rows never soft-deleted
Audit columns	created_at, updated_at, created_by, updated_by on every table; triggers or ORM hooks maintain updated_at
JSONB usage	settings, snapshots and flexible payloads only - never for relational facts that need FK integrity

Table 18.1 - Database conventions (binding for all migrations)

18.2 Standard Column Block

Every tenant-owned table ends with the following block (shown here once; the DDL in Chapter 19 repeats it verbatim in every table):

```
-- standard tenant-owned column block (repeated in every tenant table)
created_at  timestampz NOT NULL DEFAULT now(),
updated_at  timestampz NOT NULL DEFAULT now(),
created_by  uuid,                                -- FK added in the 19.10 pass
updated_by  uuid,
deleted_at  timestampz                            -- NULL = live row (soft delete)
);
```

Plus the standard tenant index: CREATE INDEX ix_{table}_tenant ON {table} (tenant_id) - and every hot list path declares its composite index explicitly in the DDL. Platform-plane tables (plans, feature_flags, geo_reference, audit sink metadata) omit tenant_id and say so in a header comment.

18.3 Domain-Level ERD - Master Data

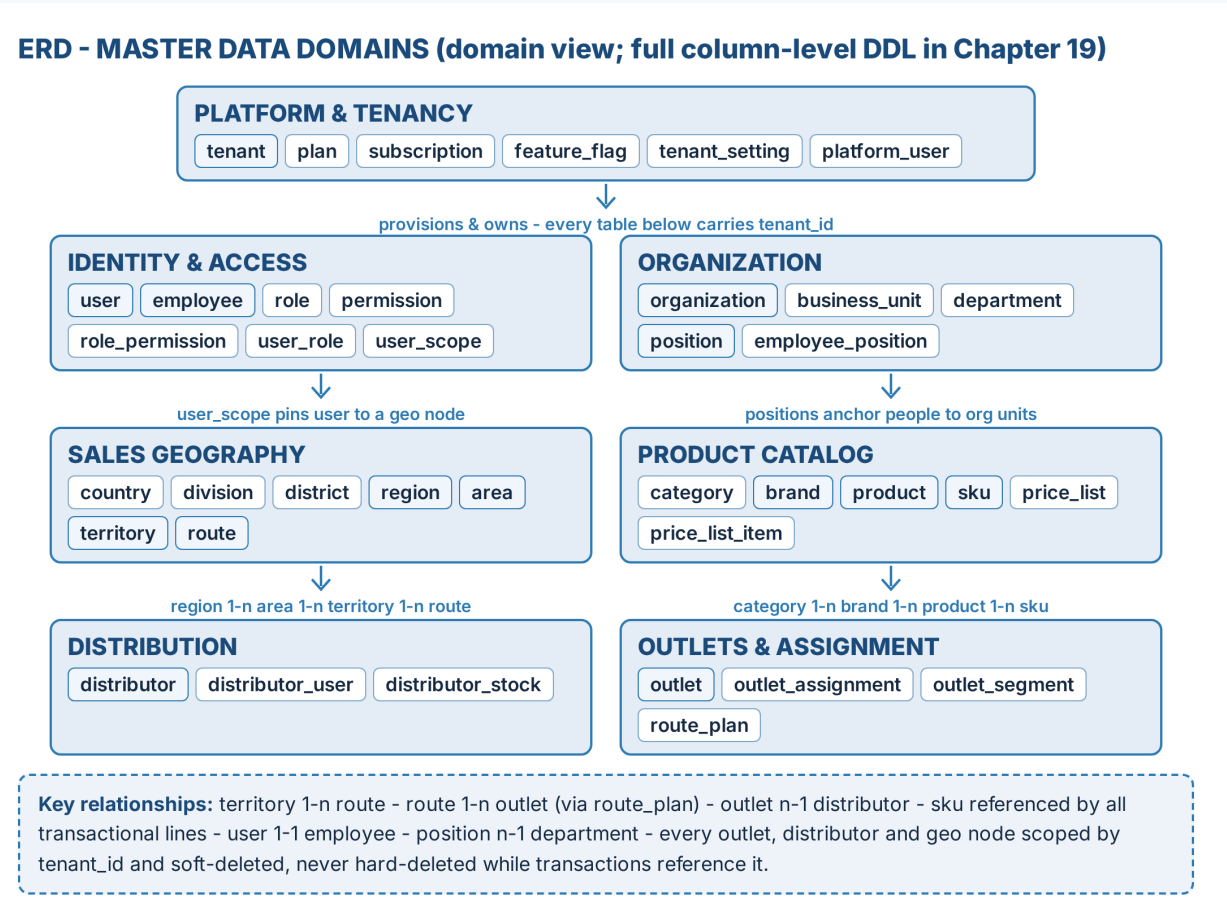


Figure 18.1 - Master data domains: platform/tenancy, identity, organization, geography, distribution, outlets, products

18.4 Domain-Level ERD - Operational Domains

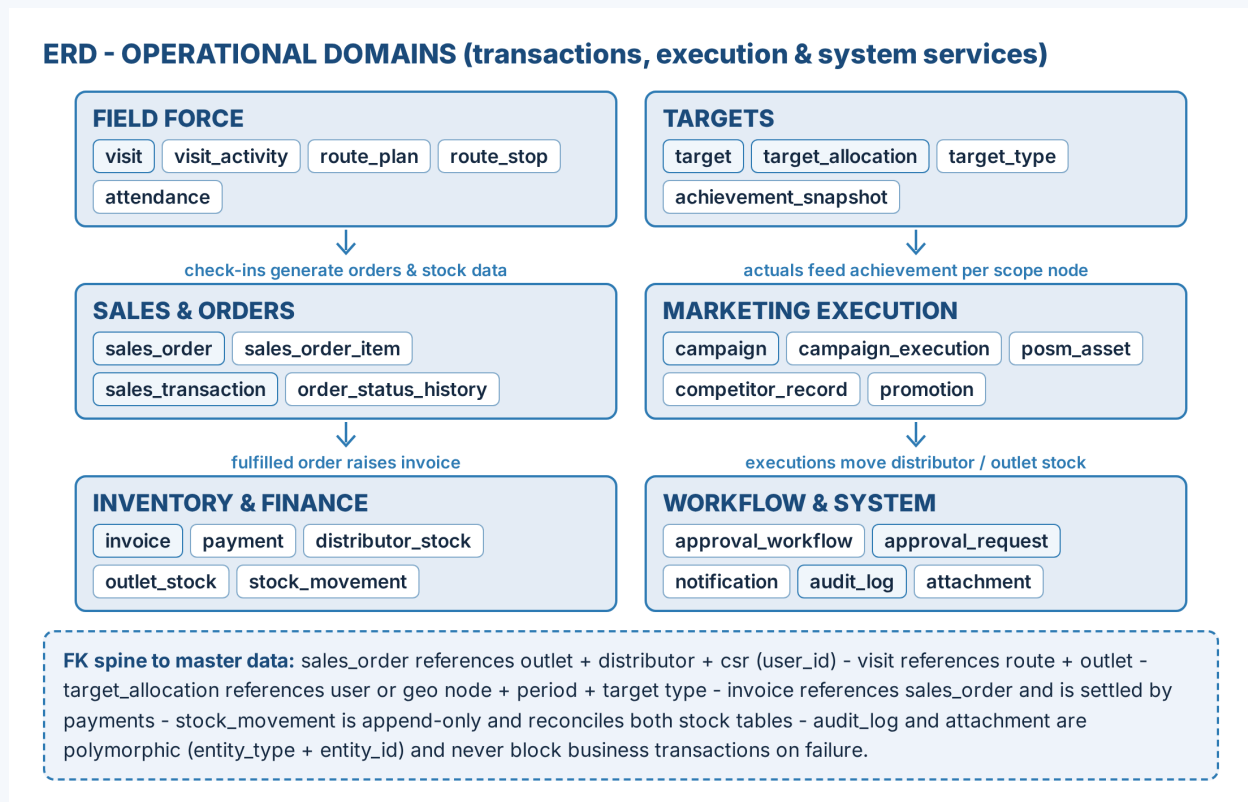


Figure 18.2 - Operational domains: field force, targets, sales, marketing, inventory, finance, workflow and system

18.5 Table Inventory by Module

Module	Tables
Platform & Tenancy	tenants, plans, subscriptions, feature_flags, tenant_features, tenant_settings
Identity & Access	users, employees, roles, permissions, role_permissions, user_roles, user_scopes
Organization	org_units, positions, employee_positions
Geography	geo_reference, geo_nodes
Distribution & Outlets	distributors, distributor_users, outlets, outlet_assignments
Products	categories, brands, products, skus, price_lists, price_list_items
Field Force	route_plans, route_stops, visits, visit_activities, attendance
Sales	sales_orders, sales_order_items, sales_transactions, order_status_history
Targets	targets, target_allocations, achievement_snapshots
Marketing	campaigns, campaign_executions, posm_assets, competitor_records
Inventory & Finance	stock_movements, distributor_stock, outlet_stock, invoices, payments
Workflow & System	approval_workflows, approval_requests, notifications, audit_logs, attachments

Table 18.2 - The 54 tables mapped to owning modules (column-level DDL: Chapter 19)

CHAPTER 19

Complete SQL DDL

This chapter is the column-level contract for all fifty-four tables. It uses PostgreSQL dialect; any mature relational engine can express the same design (uuid via the platform's key generator, JSONB via a JSON/JSONB column or a settings side table). Columns follow the conventions of Section 18.1: uuid primary keys, tenant_id on every tenant-owned table, bigint minor-unit money, timestamptz time, soft delete via deleted_at, and the standard audit column block repeated verbatim in each table.

Migration rule for the build agent

Ship the DDL as ordered, idempotent migrations in this exact group order: platform/tenancy, identity, organization/geography, distribution/outlets, product, field force, sales/targets/marketing, inventory/finance/system. Never edit a shipped migration; always add a new one. Seed rows (roles, permissions, feature flags, reference geography) are data migrations that follow the schema.

19.1 Reading the Listings

Each listing below is a complete, runnable migration fragment for its group in the exact order the build agent must ship it. CHECK constraints encode the status vocabularies fixed by this specification; index lines following each table are part of the contract and implement the hot paths named in the module chapters. Audit columns appear without inline foreign keys and are wired by the cross-cutting pass in Section 19.10 so that the table order stays linear.

19.2 Platform & Tenancy

```
-- =====
-- 19.2 PLATFORM & TENANCY (plans/feature_flags are platform-plane: no tenant_id)
-- =====
CREATE TABLE tenants (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  slug        text NOT NULL,
  status       text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN ('PENDING', 'ACTIVE', 'SUSPENDED', 'CLOSED')),
  currency     text NOT NULL DEFAULT 'BDT',
  timezone     text NOT NULL DEFAULT 'Asia/Dhaka',
  locale       text NOT NULL DEFAULT 'en',
  fiscal_start_month int NOT NULL DEFAULT 1,
  created_at   timestamptz NOT NULL DEFAULT now(),
  updated_at   timestamptz NOT NULL DEFAULT now(),
  created_by   uuid,
  updated_by   uuid,
  deleted_at   timestamptz
              -- FK added in 19.10 cross-cutting pass
              -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_tenants_slug ON tenants (slug);
CREATE TABLE plans (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code        text NOT NULL,
```

```

    name          text NOT NULL,
    limits         jsonb NOT NULL,          -- users, outlets, storage, api_rate
    price_monthly  bigint NOT NULL,         -- minor units,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,
    updated_by     uuid
);
CREATE UNIQUE INDEX uq_plans_code ON plans (code);
CREATE TABLE subscriptions (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id      uuid NOT NULL REFERENCES tenants(id),
    plan_id        uuid NOT NULL REFERENCES plans(id),
    status         text NOT NULL DEFAULT 'ACTIVE'
                CHECK (status IN ('TRIAL', 'ACTIVE', 'PAST_DUE', 'CANCELLED')),
    period_start   date NOT NULL,
    period_end     date NOT NULL,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                    -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz              -- NULL = live row (soft delete)
);
CREATE INDEX ix_subscriptions_tenant ON subscriptions (tenant_id, status);
CREATE TABLE feature_flags (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    key            text NOT NULL,          -- e.g. module.marketing
    description    text NOT NULL,
    default_on     boolean NOT NULL DEFAULT false,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,
    updated_by     uuid
);
CREATE UNIQUE INDEX uq_feature_flags_key ON feature_flags (key);
CREATE TABLE tenant_features (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id      uuid NOT NULL REFERENCES tenants(id),
    flag_id        uuid NOT NULL REFERENCES feature_flags(id),
    enabled        boolean NOT NULL DEFAULT false,
    settings       jsonb,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                    -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz              -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_tenant_features ON tenant_features (tenant_id, flag_id);
CREATE TABLE tenant_settings (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id      uuid NOT NULL UNIQUE REFERENCES tenants(id),
    settings       jsonb NOT NULL          -- currency, tz, fiscal, visit radius,
                -- approval thresholds, notification prefs,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                    -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz              -- NULL = live row (soft delete)
);

```

Listing 19.1 - Tenancy, plans, subscriptions, feature flags and settings

19.3 Identity & Access

```
-- =====
-- 19.3 IDENTITY & ACCESS
-- =====

CREATE TABLE users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid REFERENCES tenants(id), -- NULL = platform user
  user_type   text NOT NULL DEFAULT 'TENANT'
              CHECK (user_type IN ('PLATFORM','TENANT','DISTRIBUTOR')),
  email       text,
  phone       text,
  password_hash text NOT NULL,
  full_name   text NOT NULL,
  avatar_key  text,
  mfa_secret  text,
  status      text NOT NULL DEFAULT 'INVITED'
              CHECK (status IN ('INVITED','ACTIVE','DISABLED','LOCKED')),
  last_login_at timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid, -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_users_tenant_email ON users (tenant_id, email)
  WHERE tenant_id IS NOT NULL AND deleted_at IS NULL;
CREATE UNIQUE INDEX uq_users_platform_email ON users (email)
  WHERE tenant_id IS NULL AND deleted_at IS NULL;
CREATE UNIQUE INDEX uq_users_tenant_phone ON users (tenant_id, phone)
  WHERE tenant_id IS NOT NULL AND phone IS NOT NULL AND deleted_at IS NULL;
CREATE TABLE employees (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  employee_code text NOT NULL,
  full_name   text NOT NULL,
  photo_key   text,
  gender      text CHECK (gender IN ('MALE','FEMALE','OTHER')),
  date_of_birth date,
  national_id  text,
  phone       text NOT NULL,
  email       text,
  address     text,
  joining_date date NOT NULL,
  user_id     uuid UNIQUE REFERENCES users(id),
  status      text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN ('ACTIVE','INACTIVE','SUSPENDED',
                                'TERMINATED','ON_LEAVE')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid, -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_employees_code ON employees (tenant_id, employee_code)
  WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX uq_employees_phone ON employees (tenant_id, phone)
  WHERE deleted_at IS NULL;
CREATE TABLE roles (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  code        text NOT NULL, -- e.g. AREA_MANAGER
  name        text NOT NULL,
  family      text NOT NULL, -- SALES / MARKETING / DIST / EXEC / ANALYTICS
```

```

    is_system      boolean NOT NULL DEFAULT false,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                                -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz                            -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_roles_code ON roles (tenant_id, code) WHERE deleted_at IS NULL;
CREATE TABLE permissions (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    code           text NOT NULL,                        -- module.action, e.g. sales.order.approve
    module         text NOT NULL,
    description    text,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,
    updated_by     uuid
);
CREATE UNIQUE INDEX uq_permissions_code ON permissions (code);
CREATE TABLE role_permissions (
    role_id        uuid NOT NULL REFERENCES roles(id),
    permission_id  uuid NOT NULL REFERENCES permissions(id),
    PRIMARY KEY (role_id, permission_id),
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,
    updated_by     uuid
);
CREATE TABLE user_roles (
    user_id        uuid NOT NULL REFERENCES users(id),
    role_id        uuid NOT NULL REFERENCES roles(id),
    PRIMARY KEY (user_id, role_id),
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                                -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz                            -- NULL = live row (soft delete)
);
CREATE TABLE user_scopes (
    id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id      uuid NOT NULL REFERENCES tenants(id),
    user_id        uuid NOT NULL REFERENCES users(id),
    scope_type     text NOT NULL
                    CHECK (scope_type IN ('TENANT','DIVISION','REGION','AREA',
                                          'TERRITORY','ROUTE','OUTLET','DISTRIBUTOR')),
    scope_id       uuid NOT NULL, -- geo_nodes.id / distributors.id / outlets.id
    is_active      boolean NOT NULL DEFAULT true,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                                -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz                            -- NULL = live row (soft delete)
);
CREATE INDEX ix_user_scopes_user ON user_scopes (user_id) WHERE is_active;

```

Listing 19.2 - Users, employees, roles, permissions, bindings and scopes

19.4 Organization & Geography

```

-- =====
-- 19.4 ORGANIZATION & GEOGRAPHY
-- =====
CREATE TABLE org_units (

```

```

    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   uuid NOT NULL REFERENCES tenants(id),
    unit_type   text NOT NULL
                CHECK (unit_type IN ('ORGANIZATION','BUSINESS_UNIT','DEPARTMENT')),
    parent_id   uuid REFERENCES org_units(id),
    name        text NOT NULL,
    code        text,
    created_at  timestampz NOT NULL DEFAULT now(),
    updated_at  timestampz NOT NULL DEFAULT now(),
    created_by  uuid,
    updated_by  uuid,
    deleted_at  timestampz
                -- FK added in 19.10 cross-cutting pass
                -- NULL = live row (soft delete)
);
CREATE INDEX ix_org_units_tenant_parent ON org_units (tenant_id, parent_id);
CREATE TABLE positions (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   uuid NOT NULL REFERENCES tenants(id),
    org_unit_id uuid NOT NULL REFERENCES org_units(id),
    title       text NOT NULL,
                -- "Area Manager, South Chattogram"
    reports_to  uuid REFERENCES positions(id),
    default_role_id uuid REFERENCES roles(id),
    is_active   boolean NOT NULL DEFAULT true,
    created_at  timestampz NOT NULL DEFAULT now(),
    updated_at  timestampz NOT NULL DEFAULT now(),
    created_by  uuid,
    updated_by  uuid,
    deleted_at  timestampz
                -- FK added in 19.10 cross-cutting pass
                -- NULL = live row (soft delete)
);
CREATE INDEX ix_positions_unit ON positions (org_unit_id) WHERE is_active;
CREATE INDEX ix_positions_reports ON positions (reports_to);
CREATE TABLE employee_positions (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   uuid NOT NULL REFERENCES tenants(id),
    employee_id uuid NOT NULL REFERENCES employees(id),
    position_id uuid NOT NULL REFERENCES positions(id),
    is_primary   boolean NOT NULL DEFAULT true,
    valid_from   date NOT NULL,
    valid_to     date,
    created_at  timestampz NOT NULL DEFAULT now(),
    updated_at  timestampz NOT NULL DEFAULT now(),
    created_by  uuid,
    updated_by  uuid,
    deleted_at  timestampz
                -- FK added in 19.10 cross-cutting pass
                -- NULL = live row (soft delete)
);
CREATE INDEX ix_emp_pos_employee ON employee_positions (employee_id, valid_from);
CREATE TABLE geo_reference (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    level       text NOT NULL
                CHECK (level IN ('COUNTRY','DIVISION','DISTRICT','UPAZILA','CITY')),
    parent_id   uuid REFERENCES geo_reference(id),
    code        text,
    name        text NOT NULL,
    name_bn     text,
    created_at  timestampz NOT NULL DEFAULT now(),
    updated_at  timestampz NOT NULL DEFAULT now(),
    created_by  uuid,
    updated_by  uuid
);
CREATE INDEX ix_geo_reference_parent ON geo_reference (parent_id);
CREATE TABLE geo_nodes (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id   uuid NOT NULL REFERENCES tenants(id),
    level       text NOT NULL CHECK (level IN ('REGION','AREA','TERRITORY','ROUTE')),
    parent_id   uuid REFERENCES geo_nodes(id),
    path        text NOT NULL,
                -- materialized "{id}/{id}/..."

```

```

code          text NOT NULL,
name          text NOT NULL,
ref_geo_id    uuid REFERENCES geo_reference(id) -- optional mapping,
created_at    timestampz NOT NULL DEFAULT now(),
updated_at    timestampz NOT NULL DEFAULT now(),
created_by    uuid,                          -- FK added in 19.10 cross-cutting pass
updated_by    uuid,
deleted_at    timestampz                      -- NULL = live row (soft delete)
);
CREATE INDEX ix_geo_nodes_tenant_path ON geo_nodes (tenant_id, path);
CREATE INDEX ix_geo_nodes_parent ON geo_nodes (parent_id);
CREATE UNIQUE INDEX uq_geo_nodes_code ON geo_nodes (tenant_id, level, code)
WHERE deleted_at IS NULL;

```

Listing 19.3 - Org units, positions, assignments, reference geography, sales geography

19.5 Distribution & Outlets

```

-- =====
-- 19.5 DISTRIBUTION & OUTLETS
-- =====
CREATE TABLE distributors (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  distributor_code text NOT NULL,
  name        text NOT NULL,
  owner_name  text NOT NULL,
  phone       text NOT NULL,
  email       text,
  address     text,
  latitude    numeric(9,6),
  longitude    numeric(9,6),
  territory_id uuid NOT NULL REFERENCES geo_nodes(id),
  credit_limit bigint NOT NULL DEFAULT 0,
  opening_balance bigint NOT NULL DEFAULT 0,
  payment_terms_days int NOT NULL DEFAULT 0,
  status       text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN ('ACTIVE', 'INACTIVE', 'SUSPENDED', 'TERMINATED')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                          -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz                      -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_distributors_code ON distributors (tenant_id, distributor_code)
WHERE deleted_at IS NULL;
CREATE INDEX ix_distributors_territory ON distributors (territory_id);
CREATE TABLE distributor_users (
  distributor_id uuid NOT NULL REFERENCES distributors(id),
  user_id        uuid NOT NULL UNIQUE REFERENCES users(id),
  role           text NOT NULL
              CHECK (role IN ('ADMIN', 'MANAGER', 'STAFF')),
  created_at     timestampz NOT NULL DEFAULT now(),
  updated_at     timestampz NOT NULL DEFAULT now(),
  created_by     uuid,
  updated_by     uuid
);
CREATE TABLE outlets (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  outlet_code text NOT NULL,
  name        text NOT NULL,
  owner_name  text,

```

```

    phone          text,
    address        text,
    latitude       numeric(9,6) NOT NULL,
    longitude      numeric(9,6) NOT NULL,
    outlet_type    text NOT NULL CHECK (outlet_type IN
        ('RETAIL_SHOP','WHOLESALE','SUPER_SHOP','CONVENIENCE',
        'TEA_STALL','OTHER')),
    channel        text,
    segment        text CHECK (segment IN ('A','B','C','D')),
    territory_id   uuid NOT NULL REFERENCES geo_nodes(id),
    distributor_id uuid REFERENCES distributors(id),
    status         text NOT NULL DEFAULT 'ACTIVE'
        CHECK (status IN ('ACTIVE','INACTIVE','BLACKLISTED','POTENTIAL')),
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_outlets_code ON outlets (tenant_id, outlet_code)
    WHERE deleted_at IS NULL;
CREATE INDEX ix_outlets_territory ON outlets (tenant_id, territory_id);
CREATE INDEX ix_outlets_distributor ON outlets (distributor_id);
CREATE TABLE outlet_assignments (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id       uuid NOT NULL REFERENCES tenants(id),
    outlet_id       uuid NOT NULL REFERENCES outlets(id),
    user_id         uuid NOT NULL REFERENCES users(id),
    route_id        uuid REFERENCES geo_nodes(id),
    assigned_from   date NOT NULL,
    assigned_to     date,
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    created_by      uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by      uuid,
    deleted_at      timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_outlet_assignments_user ON outlet_assignments (user_id, assigned_from);
CREATE INDEX ix_outlet_assignments_outlet ON outlet_assignments (outlet_id);

```

Listing 19.4 - Distributors, portal users, outlets, assignments

19.6 Product Catalog

```

-- =====
-- 19.6 PRODUCT CATALOG
-- =====
CREATE TABLE categories (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id       uuid NOT NULL REFERENCES tenants(id),
    parent_id       uuid REFERENCES categories(id),
    name            text NOT NULL,
    code            text NOT NULL,
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    created_by      uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by      uuid,
    deleted_at      timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_categories_code ON categories (tenant_id, code)
    WHERE deleted_at IS NULL;
CREATE TABLE brands (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

tenant_id    uuid NOT NULL REFERENCES tenants(id),
category_id  uuid NOT NULL REFERENCES categories(id),
name         text NOT NULL,
code         text NOT NULL,
logo_key     text,
created_at   timestampz NOT NULL DEFAULT now(),
updated_at   timestampz NOT NULL DEFAULT now(),
created_by   uuid,                      -- FK added in 19.10 cross-cutting pass
updated_by   uuid,
deleted_at   timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_brands_code ON brands (tenant_id, code) WHERE deleted_at IS NULL;
CREATE TABLE products (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  brand_id    uuid NOT NULL REFERENCES brands(id),
  name        text NOT NULL,
  code        text NOT NULL,
  description  text,
  status       text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN ('ACTIVE','INACTIVE')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                      -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_products_code ON products (tenant_id, code)
  WHERE deleted_at IS NULL;
CREATE TABLE skus (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  product_id  uuid NOT NULL REFERENCES products(id),
  sku_code    text NOT NULL,
  name        text NOT NULL,
  pack_size   text NOT NULL,
  unit        text NOT NULL,
  barcode     text,
  mrp         bigint NOT NULL,
  selling_price bigint NOT NULL,
  cost_price  bigint,
  tax_rate    numeric(5,2) NOT NULL DEFAULT 0,
  order_multiple int NOT NULL DEFAULT 1,
  image_key   text,
  status       text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','INACTIVE')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                      -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_skus_code ON skus (tenant_id, sku_code) WHERE deleted_at IS NULL;
CREATE INDEX ix_skus_product ON skus (product_id);
CREATE TABLE price_lists (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  name        text NOT NULL,
  territory_id uuid REFERENCES geo_nodes(id), -- NULL = tenant-wide
  effective_from date NOT NULL,
  effective_to  date,
  priority      int NOT NULL DEFAULT 0,
  is_default    boolean NOT NULL DEFAULT false,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now(),
  created_by    uuid,                      -- FK added in 19.10 cross-cutting pass

```

```

    updated_by  uuid,
    deleted_at  timestampz          -- NULL = live row (soft delete)
);
CREATE INDEX ix_price_lists_tenant ON price_lists (tenant_id, effective_from);
CREATE TABLE price_list_items (
    price_list_id uuid NOT NULL REFERENCES price_lists(id),
    sku_id        uuid NOT NULL REFERENCES skus(id),
    price         bigint NOT NULL,
    mrp           bigint,
    min_qty       int NOT NULL DEFAULT 1,
    PRIMARY KEY (price_list_id, sku_id),
    created_at    timestampz NOT NULL DEFAULT now(),
    updated_at    timestampz NOT NULL DEFAULT now(),
    created_by    uuid,             -- FK added in 19.10 cross-cutting pass
    updated_by    uuid,
    deleted_at    timestampz        -- NULL = live row (soft delete)
);

```

Listing 19.5 - Categories, brands, products, SKUs, price lists

19.7 Field Force

```

-- =====
-- 19.7 FIELD FORCE
-- =====
CREATE TABLE route_plans (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    route_id      uuid NOT NULL REFERENCES geo_nodes(id),  -- level = ROUTE
    user_id       uuid NOT NULL REFERENCES users(id),
    pattern       text NOT NULL CHECK (pattern IN ('WEEKLY', 'INTERVAL', 'ADHOC')),
    weekday       smallint CHECK (weekday BETWEEN 0 AND 6),
    interval_days int,
    start_date    date NOT NULL,
    end_date      date,
    is_active     boolean NOT NULL DEFAULT true,
    created_at    timestampz NOT NULL DEFAULT now(),
    updated_at    timestampz NOT NULL DEFAULT now(),
    created_by    uuid,             -- FK added in 19.10 cross-cutting pass
    updated_by    uuid,
    deleted_at    timestampz        -- NULL = live row (soft delete)
);
CREATE INDEX ix_route_plans_user ON route_plans (user_id, start_date, is_active);
CREATE INDEX ix_route_plans_route ON route_plans (route_id);
CREATE TABLE route_stops (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    route_id      uuid NOT NULL REFERENCES geo_nodes(id),
    outlet_id     uuid NOT NULL REFERENCES outlets(id),
    sequence      int NOT NULL,
    planned_day   date,
    UNIQUE (route_id, sequence),
    created_at    timestampz NOT NULL DEFAULT now(),
    updated_at    timestampz NOT NULL DEFAULT now(),
    created_by    uuid,             -- FK added in 19.10 cross-cutting pass
    updated_by    uuid,
    deleted_at    timestampz        -- NULL = live row (soft delete)
);
CREATE TABLE visits (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    route_plan_id uuid REFERENCES route_plans(id),
    route_stop_id uuid REFERENCES route_stops(id),

```

```

    csr_id          uuid NOT NULL REFERENCES users(id),
    outlet_id       uuid NOT NULL REFERENCES outlets(id),
    visit_date      date NOT NULL,
    checkin_at      timestampz,
    checkout_at     timestampz,
    checkin_lat     numeric(9,6),
    checkin_lng     numeric(9,6),
    distance_m      int,
    radius_ok       boolean NOT NULL DEFAULT true,
    status          text NOT NULL DEFAULT 'IN_PROGRESS'
                  CHECK (status IN ('PLANNED', 'IN_PROGRESS', 'COMPLETED',
                                   'MISSED', 'CANCELLED')),

    summary         jsonb,
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    created_by      uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by      uuid,
    deleted_at      timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_visits_csr_date ON visits (csr_id, visit_date);
CREATE INDEX ix_visits_outlet_date ON visits (outlet_id, visit_date);
CREATE INDEX ix_visits_tenant_date ON visits (tenant_id, visit_date);
CREATE TABLE visit_activities (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id       uuid NOT NULL REFERENCES tenants(id),
    visit_id        uuid NOT NULL REFERENCES visits(id),
    client_uuid     uuid NOT NULL,             -- idempotent offline sync key
    activity_type   text NOT NULL CHECK (activity_type IN
    ('STOCK_CHECK', 'ORDER', 'COMPETITOR', 'POSM', 'COLLECTION', 'NOTE')),
    payload         jsonb NOT NULL,
    captured_at     timestampz NOT NULL,
    UNIQUE (tenant_id, client_uuid),
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    created_by      uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by      uuid,
    deleted_at      timestampz                 -- NULL = live row (soft delete)
);
CREATE TABLE attendance (
    id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id       uuid NOT NULL REFERENCES tenants(id),
    user_id         uuid NOT NULL REFERENCES users(id),
    work_date       date NOT NULL,
    checkin_at      timestampz,
    checkout_at     timestampz,
    status          text NOT NULL
                  CHECK (status IN ('PRESENT', 'ABSENT', 'LEAVE', 'HALF_DAY')),
    source          text NOT NULL DEFAULT 'MOBILE' CHECK (source IN ('MOBILE', 'MANUAL')),
    UNIQUE (tenant_id, user_id, work_date),
    created_at      timestampz NOT NULL DEFAULT now(),
    updated_at      timestampz NOT NULL DEFAULT now(),
    created_by      uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by      uuid,
    deleted_at      timestampz                 -- NULL = live row (soft delete)
);

```

Listing 19.6 - Route plans, stops, visits, activities, attendance

19.8 Sales, Targets & Marketing

```

-- =====
-- 19.8 SALES, TARGETS & MARKETING
-- =====

```

```

CREATE TABLE sales_orders (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  order_number text NOT NULL,
  order_date  date NOT NULL,
  csr_id      uuid NOT NULL REFERENCES users(id),
  outlet_id   uuid NOT NULL REFERENCES outlets(id),
  distributor_id uuid NOT NULL REFERENCES distributors(id),
  visit_id    uuid REFERENCES visits(id),
  source      text NOT NULL DEFAULT 'MOBILE' CHECK (source IN ('MOBILE','WEB','SYNC')),
  status      text NOT NULL DEFAULT 'DRAFT'
              CHECK (status IN ('DRAFT','SUBMITTED','APPROVED','PROCESSING',
                                'FULFILLED','CANCELLED','REJECTED')),
  gross_total bigint NOT NULL DEFAULT 0,
  discount_total bigint NOT NULL DEFAULT 0,
  tax_total   bigint NOT NULL DEFAULT 0,
  net_total   bigint NOT NULL DEFAULT 0,
  approved_by uuid REFERENCES users(id),
  approved_at timestampz,
  cancel_reason text,
  reject_reason text,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,
  updated_by  uuid,
  deleted_at  timestampz -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_sales_orders_number ON sales_orders (tenant_id, order_number);
CREATE INDEX ix_sales_orders_csr_date ON sales_orders (csr_id, order_date);
CREATE INDEX ix_sales_orders_status ON sales_orders (tenant_id, status, order_date);
CREATE INDEX ix_sales_orders_distributor ON sales_orders (distributor_id, order_date);
CREATE TABLE sales_order_items (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  order_id    uuid NOT NULL REFERENCES sales_orders(id),
  sku_id      uuid NOT NULL REFERENCES skus(id),
  qty         int NOT NULL CHECK (qty > 0),
  free_qty    int NOT NULL DEFAULT 0,
  unit_price  bigint NOT NULL, -- snapshot at creation
  discount_amount bigint NOT NULL DEFAULT 0,
  line_total  bigint NOT NULL,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,
  updated_by  uuid,
  deleted_at  timestampz -- NULL = live row (soft delete)
);
CREATE INDEX ix_order_items_order ON sales_order_items (order_id);
CREATE INDEX ix_order_items_sku ON sales_order_items (sku_id);
CREATE TABLE sales_transactions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  txn_date    date NOT NULL,
  order_id    uuid NOT NULL REFERENCES sales_orders(id),
  order_item_id uuid NOT NULL REFERENCES sales_order_items(id),
  sku_id      uuid NOT NULL REFERENCES skus(id),
  brand_id    uuid NOT NULL REFERENCES brands(id),
  category_id uuid NOT NULL REFERENCES categories(id),
  csr_id      uuid NOT NULL REFERENCES users(id),
  outlet_id   uuid NOT NULL REFERENCES outlets(id),
  route_id    uuid NOT NULL REFERENCES geo_nodes(id),
  territory_id uuid NOT NULL REFERENCES geo_nodes(id),
  area_id     uuid NOT NULL REFERENCES geo_nodes(id),
  region_id   uuid NOT NULL REFERENCES geo_nodes(id),
  distributor_id uuid NOT NULL REFERENCES distributors(id),

```

```

    qty            int NOT NULL,
    gross_amount   bigint NOT NULL,
    discount_amount bigint NOT NULL DEFAULT 0,
    net_amount     bigint NOT NULL,
    is_return      boolean NOT NULL DEFAULT false,
    return_of      uuid REFERENCES sales_transactions(id),
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now(),
    created_by     uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by     uuid,
    deleted_at     timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_txn_tenant_date ON sales_transactions (tenant_id, txn_date);
CREATE INDEX ix_txn_territory_date ON sales_transactions (territory_id, txn_date);
CREATE INDEX ix_txn_csr_date ON sales_transactions (csr_id, txn_date);
CREATE INDEX ix_txn_sku_date ON sales_transactions (sku_id, txn_date);
CREATE INDEX ix_txn_region_date ON sales_transactions (region_id, txn_date);
CREATE TABLE order_status_history (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    order_id      uuid NOT NULL REFERENCES sales_orders(id),
    from_status   text,
    to_status     text NOT NULL,
    actor_id      uuid NOT NULL REFERENCES users(id),
    note          text,
    created_at    timestampz NOT NULL DEFAULT now(),
    updated_at    timestampz NOT NULL DEFAULT now(),
    created_by    uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by    uuid,
    deleted_at    timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_osh_order ON order_status_history (order_id, created_at);
CREATE TABLE targets (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    scope_type    text NOT NULL CHECK (scope_type IN
        ('TENANT', 'REGION', 'AREA', 'TERRITORY', 'ROUTE', 'USER')),
    scope_id      uuid NOT NULL,
    target_type   text NOT NULL CHECK (target_type IN
        ('SALES_VALUE', 'SALES_QTY', 'PRODUCT_QTY', 'COVERAGE',
        'NEW_OUTLET', 'COLLECTION', 'VISITS')),
    period_type   text NOT NULL CHECK (period_type IN
        ('DAILY', 'WEEKLY', 'MONTHLY', 'QUARTERLY', 'YEARLY')),
    period_start  date NOT NULL,
    period_end    date NOT NULL,
    value         bigint NOT NULL CHECK (value >= 0),
    sku_id        uuid REFERENCES skus(id),  -- for PRODUCT_QTY targets
    status        text NOT NULL DEFAULT 'DRAFT'
        CHECK (status IN ('DRAFT', 'PENDING_APPROVAL', 'APPROVED', 'CLOSED')),
    created_at    timestampz NOT NULL DEFAULT now(),
    updated_at    timestampz NOT NULL DEFAULT now(),
    created_by    uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by    uuid,
    deleted_at    timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_targets_scope ON targets (tenant_id, scope_type, scope_id, period_start);
CREATE TABLE target_allocations (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id     uuid NOT NULL REFERENCES tenants(id),
    target_id     uuid NOT NULL REFERENCES targets(id),
    parent_allocation_id uuid REFERENCES target_allocations(id),
    owner_user_id uuid REFERENCES users(id),  -- for USER scope leaf
    geo_node_id   uuid REFERENCES geo_nodes(id),
    method        text NOT NULL CHECK (method IN ('EQUAL', 'WEIGHTED', 'MANUAL')),
    value         bigint NOT NULL CHECK (value >= 0),

```

```

    version          int NOT NULL DEFAULT 1,
    allocated_by     uuid NOT NULL REFERENCES users(id),
    approved_by      uuid REFERENCES users(id),
    created_at       timestampz NOT NULL DEFAULT now(),
    updated_at       timestampz NOT NULL DEFAULT now(),
    created_by       uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by       uuid,
    deleted_at       timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_allocations_target ON target_allocations (target_id, version);
CREATE INDEX ix_allocations_owner ON target_allocations (owner_user_id);
CREATE TABLE achievement_snapshots (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         uuid NOT NULL REFERENCES tenants(id),
    scope_type        text NOT NULL,
    scope_id          uuid NOT NULL,
    target_id         uuid REFERENCES targets(id),
    period_type       text NOT NULL,
    period_date       date NOT NULL,
    target_value      bigint NOT NULL DEFAULT 0,
    actual_value      bigint NOT NULL DEFAULT 0,
    computed_at       timestampz NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, scope_type, scope_id, target_id, period_type, period_date),
    created_at        timestampz NOT NULL DEFAULT now(),
    updated_at        timestampz NOT NULL DEFAULT now(),
    created_by        uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by        uuid,
    deleted_at        timestampz                 -- NULL = live row (soft delete)
);
CREATE TABLE campaigns (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         uuid NOT NULL REFERENCES tenants(id),
    code              text NOT NULL,
    name              text NOT NULL,
    brand_id          uuid NOT NULL REFERENCES brands(id),
    objective          text NOT NULL CHECK (objective IN
        ('VISIBILITY', 'VOLUME', 'ACTIVATION', 'LAUNCH')),
    start_date        date NOT NULL,
    end_date          date NOT NULL,
    budget            bigint NOT NULL DEFAULT 0,
    spent             bigint NOT NULL DEFAULT 0,
    expected_outlets  int NOT NULL DEFAULT 0,
    geo_scope         jsonb,                     -- targeted regions/areas/territories
    status            text NOT NULL DEFAULT 'DRAFT'
        CHECK (status IN ('DRAFT', 'PLANNED', 'APPROVED', 'ACTIVE',
            'PAUSED', 'COMPLETED', 'CANCELLED')),
    created_at        timestampz NOT NULL DEFAULT now(),
    updated_at        timestampz NOT NULL DEFAULT now(),
    created_by        uuid,                      -- FK added in 19.10 cross-cutting pass
    updated_by        uuid,
    deleted_at        timestampz                 -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_campaigns_code ON campaigns (tenant_id, code) WHERE deleted_at IS NULL;
CREATE INDEX ix_campaigns_status ON campaigns (tenant_id, status, start_date);
CREATE TABLE campaign_executions (
    id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id         uuid NOT NULL REFERENCES tenants(id),
    campaign_id       uuid NOT NULL REFERENCES campaigns(id),
    outlet_id         uuid NOT NULL REFERENCES outlets(id),
    status            text NOT NULL DEFAULT 'PENDING'
        CHECK (status IN ('PENDING', 'EXECUTED', 'SKIPPED', 'FAILED')),
    executed_by       uuid REFERENCES users(id),
    executed_at       timestampz,
    visit_id          uuid REFERENCES visits(id),
    photo_attachment_id uuid REFERENCES attachments(id),

```

```

    checklist    jsonb,
    UNIQUE (campaign_id, outlet_id),
    created_at   timestampz NOT NULL DEFAULT now(),
    updated_at   timestampz NOT NULL DEFAULT now(),
    created_by   uuid,                -- FK added in 19.10 cross-cutting pass
    updated_by   uuid,
    deleted_at   timestampz           -- NULL = live row (soft delete)
);
CREATE TABLE posm_assets (
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid NOT NULL REFERENCES tenants(id),
    name         text NOT NULL,
    asset_type   text NOT NULL CHECK (asset_type IN
        ('POSTER', 'DANGLER', 'SHELF_STRIP', 'DISPLAY_UNIT', 'OTHER')),
    brand_id     uuid REFERENCES brands(id),
    qty_total    int NOT NULL DEFAULT 0,
    qty_distributed int NOT NULL DEFAULT 0,
    unit_cost    bigint NOT NULL DEFAULT 0,
    created_at   timestampz NOT NULL DEFAULT now(),
    updated_at   timestampz NOT NULL DEFAULT now(),
    created_by   uuid,                -- FK added in 19.10 cross-cutting pass
    updated_by   uuid,
    deleted_at   timestampz           -- NULL = live row (soft delete)
);
CREATE TABLE competitor_records (
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid NOT NULL REFERENCES tenants(id),
    visit_id     uuid REFERENCES visits(id),
    outlet_id    uuid NOT NULL REFERENCES outlets(id),
    competitor_brand text NOT NULL,
    competitor_product text,
    pack_size    text,
    price        bigint,
    availability  text NOT NULL
        CHECK (availability IN ('IN_STOCK', 'OUT_OF_STOCK', 'NOT_CARRIED')),
    promotion    text,
    photo_attachment_id uuid REFERENCES attachments(id),
    notes        text,
    observed_at   date NOT NULL,
    recorded_by   uuid NOT NULL REFERENCES users(id),
    created_at   timestampz NOT NULL DEFAULT now(),
    updated_at   timestampz NOT NULL DEFAULT now(),
    created_by   uuid,                -- FK added in 19.10 cross-cutting pass
    updated_by   uuid,
    deleted_at   timestampz           -- NULL = live row (soft delete)
);
CREATE INDEX ix_competitor_outlet ON competitor_records (outlet_id, observed_at);
CREATE INDEX ix_competitor_brand ON competitor_records (tenant_id, competitor_brand,
    observed_at);

```

Listing 19.7 - Orders, items, transactions, history, targets, allocations, campaigns, competitor records

19.9 Inventory, Finance & System Services

```

-- =====
-- 19.9 INVENTORY, FINANCE & SYSTEM SERVICES
-- =====
CREATE TABLE stock_movements (
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid NOT NULL REFERENCES tenants(id),
    movement_type text NOT NULL CHECK (movement_type IN
        ('RECEIVED', 'SOLD', 'ADJUST', 'TRANSFER', 'RETURN')),
    sku_id       uuid NOT NULL REFERENCES skus(id),

```

```

distributor_id uuid REFERENCES distributors(id),
outlet_id      uuid REFERENCES outlets(id),
qty           int NOT NULL,
ref_type      text,
ref_id        uuid,
moved_at      timestampz NOT NULL DEFAULT now(),
note          text,
created_at    timestampz NOT NULL DEFAULT now(),
updated_at    timestampz NOT NULL DEFAULT now(),
created_by    uuid,                -- FK added in 19.10 cross-cutting pass
updated_by    uuid,
deleted_at    timestampz          -- NULL = live row (soft delete)
);
CREATE INDEX ix_stock_moves_distributor ON stock_movements (distributor_id, sku_id, moved_at);
CREATE INDEX ix_stock_moves_outlet ON stock_movements (outlet_id, sku_id, moved_at);
CREATE TABLE distributor_stock (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  distributor_id uuid NOT NULL REFERENCES distributors(id),
  sku_id      uuid NOT NULL REFERENCES skus(id),
  qty         int NOT NULL DEFAULT 0 CHECK (qty >= 0),
  UNIQUE (distributor_id, sku_id),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz          -- NULL = live row (soft delete)
);
CREATE TABLE outlet_stock (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  outlet_id   uuid NOT NULL REFERENCES outlets(id),
  sku_id      uuid NOT NULL REFERENCES skus(id),
  qty         int NOT NULL,
  observed_at timestampz NOT NULL,
  visit_id    uuid REFERENCES visits(id),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz          -- NULL = live row (soft delete)
);
CREATE INDEX ix_outlet_stock_outlet ON outlet_stock (outlet_id, sku_id, observed_at);
CREATE TABLE invoices (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  invoice_number text NOT NULL,
  order_id    uuid NOT NULL UNIQUE REFERENCES sales_orders(id),
  distributor_id uuid NOT NULL REFERENCES distributors(id),
  invoice_date date NOT NULL,
  due_date    date NOT NULL,
  amount      bigint NOT NULL,
  tax_amount  bigint NOT NULL DEFAULT 0,
  total_amount bigint NOT NULL,
  paid_amount  bigint NOT NULL DEFAULT 0,
  status       text NOT NULL DEFAULT 'UNPAID'
              CHECK (status IN ('UNPAID', 'PARTIALLY_PAID', 'PAID', 'VOID')),
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz          -- NULL = live row (soft delete)
);
CREATE UNIQUE INDEX uq_invoices_number ON invoices (tenant_id, invoice_number);
CREATE INDEX ix_invoices_distributor ON invoices (distributor_id, due_date);

```

```

CREATE TABLE payments (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  invoice_id  uuid NOT NULL REFERENCES invoices(id),
  paid_at     timestampz NOT NULL DEFAULT now(),
  amount      bigint NOT NULL CHECK (amount > 0),
  method      text NOT NULL CHECK (method IN
    ('CASH', 'CHEQUE', 'BKASH', 'NAGAD', 'BANK', 'OTHER')),
  collected_by uuid REFERENCES users(id),
  reference   text,
  is_void     boolean NOT NULL DEFAULT false,
  void_reason text,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                      -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz                 -- NULL = live row (soft delete)
);
CREATE INDEX ix_payments_invoice ON payments (invoice_id);
CREATE INDEX ix_payments_collected ON payments (collected_by, paid_at);
CREATE TABLE approval_workflows (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  entity_type text NOT NULL,             -- SALES_ORDER / TARGET / CAMPAIGN / ...
  name        text NOT NULL,
  conditions  jsonb NOT NULL,           -- eligibility, e.g. {"net_total_gt": 2500000}
  chain       jsonb NOT NULL,          -- ordered approver position/role refs
  escalation_hours int NOT NULL DEFAULT 24,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                    -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz               -- NULL = live row (soft delete)
);
CREATE INDEX ix_workflows_entity ON approval_workflows (tenant_id, entity_type);
CREATE TABLE approval_requests (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  workflow_id uuid NOT NULL REFERENCES approval_workflows(id),
  entity_type text NOT NULL,
  entity_id   uuid NOT NULL,
  requested_by uuid NOT NULL REFERENCES users(id),
  current_step int NOT NULL DEFAULT 1,
  status      text NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED', 'ESCALATED')),
  decision_log jsonb,
  due_at       timestampz,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  created_by  uuid,                    -- FK added in 19.10 cross-cutting pass
  updated_by  uuid,
  deleted_at  timestampz               -- NULL = live row (soft delete)
);
CREATE INDEX ix_approvals_pending ON approval_requests (tenant_id, status, due_at);
CREATE INDEX ix_approvals_entity ON approval_requests (entity_type, entity_id);
CREATE TABLE notifications (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   uuid NOT NULL REFERENCES tenants(id),
  user_id     uuid NOT NULL REFERENCES users(id),
  event_type  text NOT NULL,
  title       text NOT NULL,
  body        jsonb,
  channels    jsonb NOT NULL,          -- {"inapp":true,"push":true,"sms":false}
  read_at     timestampz,

```

```

    sent_at      timestampz,
    status       text NOT NULL DEFAULT 'QUEUED'
                CHECK (status IN ('QUEUED','SENT','FAILED','READ')),
    created_at   timestampz NOT NULL DEFAULT now(),
    updated_at   timestampz NOT NULL DEFAULT now(),
    created_by   uuid,                -- FK added in 19.10 cross-cutting pass
    updated_by   uuid,
    deleted_at   timestampz          -- NULL = live row (soft delete)
);
CREATE INDEX ix_notifications_user ON notifications (user_id, read_at);
CREATE TABLE audit_logs (
    -- APPEND-ONLY: application role has INSERT/SELECT only.
    -- Platform-plane actions carry tenant_id NULL.
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid REFERENCES tenants(id),
    user_id      uuid REFERENCES users(id),
    action       text NOT NULL,
    module       text NOT NULL,
    entity_type  text NOT NULL,
    entity_id    uuid,
    old_value    jsonb,
    new_value    jsonb,
    ip           inet,
    user_agent   text,
    created_at   timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_audit_tenant_time ON audit_logs (tenant_id, created_at DESC);
CREATE INDEX ix_audit_entity ON audit_logs (entity_type, entity_id);
CREATE INDEX ix_audit_user_time ON audit_logs (user_id, created_at DESC);
CREATE TABLE attachments (
    id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id    uuid NOT NULL REFERENCES tenants(id),
    entity_type  text NOT NULL,
    entity_id    uuid,
    storage_key  text NOT NULL,        -- tenant-prefixed, opaque
    file_name    text NOT NULL,
    mime_type    text NOT NULL,
    size_bytes   bigint NOT NULL,
    sha256       text NOT NULL,
    scan_status  text NOT NULL DEFAULT 'PENDING'
                CHECK (scan_status IN ('PENDING','CLEAN','FLAGGED')),
    created_at   timestampz NOT NULL DEFAULT now(),
    updated_at   timestampz NOT NULL DEFAULT now(),
    created_by   uuid,                -- FK added in 19.10 cross-cutting pass
    updated_by   uuid,
    deleted_at   timestampz          -- NULL = live row (soft delete)
);
CREATE INDEX ix_attachments_entity ON attachments (entity_type, entity_id);

```

Listing 19.8 - Stock, invoices, payments, workflows, notifications, audit, attachments

19.10 Cross-Cutting FK Pass & Tamper Enforcement

After all tables exist, a final migration adds the audit-column foreign keys mechanically for every table that has created_by/updated_by, and locks down the audit table against mutation. The mechanism (not hand-written per table) keeps the fifty-four CREATE TABLE blocks clean.

```

-- applied by migration loop over information_schema (illustrative SQL):
ALTER TABLE tenants
    ADD CONSTRAINT fk_tenants_created_by FOREIGN KEY (created_by) REFERENCES users(id);
-- ... repeated for every table with audit columns ...
-- append-only enforcement for the audit trail (MUST):

```

```

REVOKE UPDATE, DELETE ON audit_logs FROM app_write_role;
CREATE RULE audit_no_update AS ON UPDATE TO audit_logs DO INSTEAD NOTHING;
CREATE RULE audit_no_delete AS ON DELETE TO audit_logs DO INSTEAD NOTHING;

```

Listing 19.9 - Cross-cutting constraints and append-only audit enforcement

CHAPTER 20

API Standards & Complete Endpoint Catalog

The API is the only way clients touch the platform - there are no private backdoors for the web frontend. All endpoints are versioned under /api/v1, authenticated, tenant-resolved server-side, permission-guarded, scope-filtered, rate-limited and audited where required. This chapter fixes the standards, then catalogs every endpoint by module. Path parameters and query conventions are uniform, so the catalog needs no per-endpoint prose: method + path + permission + scope rule fully determine behavior.

20.1 API Standards (MUST Set)

Concern	Standard
Versioning	/api/v1/...; breaking changes ship as /api/v2 with 6-month overlap
Auth	Bearer session token or API token; tenant + user resolved server-side; never from headers the client controls
List envelope	{"data": [...], "page": 1, "per_page": 25, "total": 1043, "total_pages": 42}
Filtering	Repeated query params (?status=A&status=B), ranges via _gte/_lte suffixes, geo via scope param when permitted
Sorting	?sort=-created_at,name (leading minus = desc); sortable fields whitelisted per endpoint
Search	?q= server-side search over declared columns; no client-side filtering of large sets
Errors	{"error": {"code": "PERMISSION_DENIED", "message": "...", "details": {...}, "trace_id": "..."}} with correct HTTP status
Idempotency	Unsafe ops accept Idempotency-Key header; sync ingest requires client_uuid
Rate limiting	Per token + per IP; 429 with Retry-After; sync endpoints have separate device limits
Audit	Mutations emit audit events; exports record filters and row counts
Dates/currency	ISO-8601 UTC timestamps; money as integer minor units; BDT default currency

Table 20.1 - Platform-wide API standards

Authorization order of evaluation

Every request passes four gates in order: (1) authenticated identity, (2) permission check on the route, (3) tenant isolation of every referenced object, (4) geographic scope filter on reads and scope-fit validation on writes. Failing any gate returns 403/404 without leaking existence of out-of-reach objects, and gates 2-4 are also enforced inside background jobs and exports.

20.2 Platform & Tenancy

Method	Endpoint	Permission	Scope rule
POST	/api/v1/platform/auth/login	platform.role	Platform plane only
GET	/api/v1/platform/tenants	platform.tenant.view	Platform plane
POST	/api/v1/platform/tenants	platform.tenant.create	Provisions tenant + admin
PATCH	/api/v1/platform/tenants/{id}/status	platform.tenant.suspend	Suspend/activate, audited
PATCH	/api/v1/platform/tenants/{id}/plan	platform.tenant.plan	Change plan
PUT	/api/v1/platform/tenants/{id}/features	platform.tenant.features	Feature flags per tenant
GET	/api/v1/platform/tenants/{id}/usage	platform.tenant.view	Usage metrics
POST	/api/v1/platform/support-grants	platform.support.grant	Time-boxed tenant access, audited
GET	/api/v1/platform/audit	platform.audit.view	Platform-level audit
GET	/api/v1/tenant/profile	authenticated	Own tenant
PATCH	/api/v1/tenant/settings	tenant.settings	TENANT_ADMIN
GET/PUT	/api/v1/tenant/features	tenant.features.manage	TENANT_ADMIN (within plan)

Table 20.2 - Platform & tenancy endpoints

20.3 Identity, Organization & Geography

Method	Endpoint	Permission	Scope rule
POST	/api/v1/auth/login	public	Tenant-resolved on login
POST	/api/v1/auth/refresh logout	authenticated	Own session
POST	/api/v1/auth/forgot reset	public (token)	Signed single-use tokens
GET/POST	/api/v1/employees	employee.view / create	Scope-filtered lists
GET/PATCH	/api/v1/employees/{id}	employee.view / update	Within scope
POST	/api/v1/employees/{id}/status	employee.status	HR / TENANT_ADMIN
GET/POST	/api/v1/users	user.view / user.create	TENANT_ADMIN
PATCH	/api/v1/users/{id}/roles	user.roles	TENANT_ADMIN, audited
PATCH	/api/v1/users/{id}/scope	user.scope	TENANT_ADMIN, audited
GET/POST	/api/v1/roles	role.view / role.manage	Role CRUD + permission bundling
GET/POST	/api/v1/org/units	org.view / org.manage	Tree CRUD

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/org/positions	org.view / org.manage	Position CRUD + report-to
POST	/api/v1/org/positions/{id}/assign	org.manage	Bind employee to position
GET	/api/v1/org/chain/{positionId}	org.view	Approver chain resolution
GET/POST	/api/v1/geo/nodes	geo.view / geo.manage	Region/area/territory/route CRUD
GET	/api/v1/geo/subtree?node=	geo.view	Scope ladder subtree
GET	/api/v1/geo/reference	authenticated	Reference geography (read-only)

Table 20.3 - Identity, organization & geography endpoints

20.4 Distributors, Outlets & Products

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/distributors	distributor.view / create	Scope-filtered / admin create
GET/PATCH	/api/v1/distributors/{id}	distributor.view / update	Own entity for DIST_*
POST	/api/v1/distributors/{id}/status	distributor.status	TENANT_ADMIN
GET/POST	/api/v1/distributors/{id}/users	distributor.user.manage	DIST_ADMIN, TENANT_ADMIN
GET/POST	/api/v1/outlets	outlet.view / outlet.create	Scope-filtered; CSR proposals POTENTIAL
GET/PATCH	/api/v1/outlets/{id}	outlet.view / outlet.update	Within scope
POST	/api/v1/outlets/{id}/assign	outlet.assign	TSO upward
POST	/api/v1/outlets/import	outlet.import	Dry-run + commit
GET/POST	/api/v1/products/categories brands products	product.view / manage	Catalog CRUD
GET/POST	/api/v1/products/skus	product.view / manage	SKU CRUD
GET/POST	/api/v1/price-lists	product.price.manage	Versioned lists + items
GET	/api/v1/price-lists/resolve?outlet=&sku;=	product.view	Effective price resolution

Table 20.4 - Master data II endpoints

20.5 Field Force & Sync

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/routes	route.view / route.manage	Route + plan CRUD

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/routes/{id}/stops	route.manage	Stop sequencing
GET	/api/v1/field/my-day	CSR role	Route + stops + targets for today
POST	/api/v1/field/attendance	attendance.create	Own attendance
POST	/api/v1/field/visits/check-in	visit.create	Own route outlets; GPS validated
POST	/api/v1/field/visits/{id}/activities	visit.update	Own visit, typed activities
POST	/api/v1/field/visits/{id}/check-out	visit.update	Summary capture
POST	/api/v1/field/visits/{id}/photos	visit.update	Signed upload flow
POST	/api/v1/sync/batch	device token	Up to 200 queued events, idempotent
GET	/api/v1/sync/catalog?since=	device token	Delta: SKUs, prices, outlets, routes
GET/PATCH	/api/v1/field/attendance/{id}	attendance.view / correct	Supervisor correction, audited

Table 20.5 - Field force & sync endpoints

20.6 Sales, Targets, Marketing, Inventory, Finance

Method	Endpoint	Permission	Scope rule
GET/POST	/api/v1/sales/orders	sales.order.view / create	Scope-filtered lists
POST	/api/v1/sales/orders/{id}/submit cancel	sales.order.submit / cancel	Workflow gates
POST	/api/v1/sales/orders/{id}/approve reject	sales.order.approve	Approver per workflow
POST	/api/v1/sales/orders/{id}/process fulfill	sales.order.fulfill	DISTRIBUTOR_* roles
GET	/api/v1/sales/transactions	sales.transaction.view	Scope-filtered, server-side agg
GET/POST	/api/v1/targets	target.view / target.create	Own node rows
POST	/api/v1/targets/{id}/allocate submit approve	target.allocate / approve	Cascade + workflow
GET	/api/v1/targets/achievement	target.achievement.view	Scope rollup query
GET	/api/v1/targets/traceability/{allocationId}	target.achievement.view	Allocation chain
GET/POST	/api/v1/campaigns	campaign.view / create	Marketing chain
POST	/api/v1/campaigns/{id}/submit approve pause resume cancel	campaign.approve etc.	State machine gates

Method	Endpoint	Permission	Scope rule
GET	/api/v1/campaigns/{id}/execution	campaign.execution.view	Scoped progress + photos
POST	/api/v1/visits/{id}/trade-execution	trade.execute	CSR on own visit
POST	/api/v1/visits/{id}/competitor	competitor.create	CSR on own visit
GET	/api/v1/marketing/competitor-summary	competitor.view	Aggregated, scoped
GET/POST	/api/v1/posm	posm.manage	Asset registry + distribution
POST	/api/v1/inventory/stock-movements	stock.movement.create	DIST own / FINANCE
GET	/api/v1/inventory/distributor-stock availability	stock.view	Scoped reads
GET	/api/v1/finance/invoices	invoice.view	Own + chain
POST	/api/v1/finance/payments	payment.create	CSR field / DIST / FINANCE
GET	/api/v1/finance/outstanding	invoice.view	Aging buckets, scoped

Table 20.6 - Transactional module endpoints

20.7 Workflow, Notifications, Reports, Files, Audit

Method	Endpoint	Permission	Scope rule
GET	/api/v1/approvals/my-queue	workflow.decide	Approver-resolved queue
POST	/api/v1/approvals/{id}/approve reject delegate	workflow.decide	Current step only, audited
GET/POST	/api/v1/workflows	workflow.manage	TENANT_ADMIN configuration
GET	/api/v1/notifications	authenticated	Own notifications
POST	/api/v1/notifications/{id}/read read-all	owner	Own rows
GET/PATCH	/api/v1/notification-preferences	owner	Channel opt-ins
GET	/api/v1/reports	reports.view	Definitions by role
POST	/api/v1/reports/{key}/run export	reports.view / export	Scope injected into query
GET	/api/v1/reports/jobs/{jobId}	reports.view	Own job + artifact link
CRUD	/api/v1/reports/schedules	reports.schedule	Per-tenant schedules
POST	/api/v1/files/upload	per-entity permission	Signed flow, tenant-prefixed keys
GET	/api/v1/files/{id}/download	per-entity permission	Signed URL, re-authorized
GET	/api/v1/audit	audit.view	Tenant audit; platform via /platform/audit

Table 20.7 - System service endpoints

CHAPTER 21

Security & Non-Functional Requirements

Security requirements here are structural, not aspirational: they are the same controls Chapters 3, 4 and 19 made mandatory (server-derived tenant context, central query scope, database constraints, signed file access, append-only audit) restated as an acceptance envelope, plus the operational controls a production SaaS must carry. Non-functional targets are stated as measurable numbers so the build agent can test against them rather than assert "fast".

21.1 Security Controls

Control	Requirement (MUST unless noted)
Authentication	Argon2id/bcrypt password hashing; lockout with backoff; session fixation protection; MFA-ready flows
Authorization	Permission + scope checks server-side on every endpoint, job and export; default deny
Tenant isolation	Chapter 3.3 contract fully implemented and test-verified; client-supplied tenant ids ignored
Input validation	Schema validation on every endpoint (payload, query, path); strong typing; size caps
Output encoding	Framework-standard escaping; no raw HTML injection; CSP headers on web surfaces
CSRF	Double-submit or SameSite cookies for cookie-based sessions
File security	MIME allow-list, size caps, malware scan hook, signed short-lived URLs, tenant-prefixed keys
Secrets	Environment/secret-manager storage; never in code or repo; rotation runbook
Transport	TLS 1.2+ everywhere, HSTS, secure cookies, no mixed content
Rate limiting	Per-IP and per-token limits; stricter on auth and sync endpoints
Audit	Chapter 13.3 coverage; append-only storage; export auditing
Dependency hygiene	Locked dependency manifests; automated vulnerability scanning in CI
Support access	Platform support grants are time-boxed, reason-coded, fully audited, auto-expiring

Table 21.1 - Security control set

21.2 Threats to Controls Mapping

Threat	Control response
Cross-tenant data mining via manipulated ids	Tenant filter in central scope + composite FK constraints + isolation test suite
Privilege escalation via role tampering	Role changes admin-only, audited, effective immediately; permissions never accepted from client
Export as exfiltration channel	Exports inherit scope filters; row caps; export audit with filters and counts

Threat	Control response
Offline sync poisoning	Server-side revalidation of all queued events; idempotency by client_uuid; flagged drafts on failure
GPS spoofing / buddy punching	Radius validation, flagged out-of-radius check-ins, reason codes, supervisor review queue
Credential stuffing	Lockout + backoff, breach-list password checks, optional MFA, anomaly alerts
Invoice/payment manipulation	Immutable invoices; payments via audited methods; voiding requires permission + reason
Audit tampering	Append-only enforcement at DB role level; no UPDATE/DELETE path exists in the application

Table 21.2 - Threat model highlights and mandatory responses

21.3 Non-Functional Targets

Dimension	Target (reference scale: 50 tenants, 500 CSRs, 2M transactions/yr)
Interactive latency	p95 under 400 ms for list/detail endpoints; p95 under 1.5 s for dashboard first paint
Throughput	200 requests/s sustained; sync batch ingest 50 devices/min without queue backlog
Concurrency	500 concurrent authenticated users; 50 simultaneous CSR syncs
Availability	99.5% monthly for v1 (single region); graceful degradation for read paths
Data durability	Daily backups + PITR; RPO 15 min; RTO 2 h; restore drill each quarter
Scalability path	Stateless app tier scales horizontally; read replica for reports; partition-ready transaction tables
Data retention	Transactional data retained for the subscription life + 90 days; exports 90 days; audit 3 years
Localization	en + bn catalogs; lakh/crore BDT formatting; Asia/Dhaka default timezone

Table 21.3 - NFR envelope (measured in CI load smoke tests)

CHAPTER 22

Testing Strategy & Tenant Isolation Matrix

Testing is the build agent's proof of work. A module without its tests is not finished even if it compiles and demos well. The strategy is a standard pyramid weighted toward integration tests, with one suite elevated above all others: the tenant isolation suite, which must run on every merge and must fail the build on any violation.

22.1 Test Pyramid & Required Types

Layer	What it covers	Gate
Unit	Domain logic: pricing, target cascade math, scope subtree, metric formulas	Every module; fast; runs on commit

Layer	What it covers	Gate
Integration	Module + database: repositories, migrations up/down, transactional flows	Every module
API / contract	Endpoint behavior: auth, permissions, validation, pagination envelopes, idempotency	Every endpoint
Authorization	Permission matrix per role from Appendix B executed against endpoints	Every merge
Tenant isolation	The matrix in 22.2 - two tenants, every module, every access path	Every merge; zero tolerance
UI (smoke)	Critical paths per role: login, dashboard, order create/approve, visit sync	Nightly
Load smoke	NFR targets from 21.3 on seeded reference data	Weekly + pre-release

Table 22.1 - Test layers and merge gates

22.2 Tenant Isolation Test Matrix (zero-tolerance suite)

Fixtures: Tenant A with users A-admin, A-CSR; Tenant B with B-admin, B-CSR; one distributor and one outlet each; one order each. The suite asserts, for EVERY tenant-owned resource and EVERY endpoint family:

Operation	Asserted behavior for A-user against B-data
List / query	Never appears in any list, filter combination, or total count
Read by id	404 (not 403 - no existence leak)
Create referencing B objects	422/404; FK path impossible; attempt audited
Update	404; no partial write anywhere
Delete / cancel	404; B-data unchanged byte-for-byte
Export	Zero B rows in any format; export audit shows A scope only
File access	Signed URL for B attachment denied; key guessable? still denied
Aggregate / dashboard	B contributions excluded from every A rollup
Search / suggest	B names never returned via autocomplete or global search
Platform support grant OFF	Platform roles see zero tenant rows without an active grant

Table 22.2 - Isolation assertions applied to all 54 tables via parameterized tests

22.3 Authorization Test Cases

- For each seeded role, run the endpoint matrix from Chapter 20 and assert the permission verdict matches Appendix B; the test reads the same catalog as the seeder so drift is impossible.
- Scope ladder tests: for each role-scope pair, seed data inside, at and beyond the scope boundary; assert counts equal expected subtrees.
- Delegation and escalation tests for the approval engine, including vacant position escalation.

- Offline sync adversarial tests: replayed events, duplicate client_uuid, out-of-order timestamps, stale price submissions.

22.4 Test Data & Environment Strategy

- Seed script builds the reference tenant: ABC Tobacco with the Chapter 2 hierarchy, Chattogram geography, 3 distributors, 150 outlets, 40 SKUs, 90 days of visits and orders - used by dev, CI and demos alike.
- Every test run starts from migrated-empty + seed; no test depends on another test's leftovers.
- Anonymity rule: no production data in any non-production environment, ever.

CHAPTER 23

Definition of Done, Build Order & Phase Gates

This chapter is the build agent's operating system: the DoD checklist every module must satisfy, the master build order that sequences the work, and the discipline rules that keep 54 tables and 17 modules coherent as the codebase grows. The build order encodes dependency reality: identity before organization, geography before outlets, outlets before routes, routes before visits, visits before orders, everything before dashboards.

28 Build steps	13 DoD checks	7 Merge gates	0 Tolerance for isolation defects
--------------------------	-------------------------	-------------------------	---

23.1 Definition of Done (per module)

No.	Done means
1	Database migration shipped, reversible, indexed per Chapter 19
2	Backend service + endpoints per Chapter 20 catalog with validation
3	Authorization enforced: permission + tenant + scope on every path
4	Tenant isolation verified by the parameterized suite of Chapter 22.2
5	Frontend screens per Chapter 17 with all five UI states
6	Loading, error, empty, no-permission, offline states designed, not defaulted
7	Server-side pagination, sorting, filtering, search on every list
8	Export inherits scope filters; export action audited
9	Audit events emitted for every mutation per Chapter 13.3
10	Notifications wired for the module's events per Table 13.1
11	Tests: unit + integration + API + authorization + isolation all green

No.	Done means
12	Documentation: module README updated (entities, endpoints, rules, flags)
13	Feature flag honored: module can be disabled per tenant without errors

Table 23.1 - The 13-point Definition of Done

23.2 Master Build Order

Step	Deliverable	Depends on	Gate
01	Foundation: repo, CI, environments, migrations tooling	-	CI green on hello-world deploy
02	Multi-tenancy: tenants, settings, feature flags	01	Provision tenant E2E
03	Authentication: users, sessions, tokens, reset, logout	02	Auth suite green
04	RBAC: roles, permissions, bindings, guards	03	Authorization matrix green
05	Organization: units, positions, assignments	04	Org CRUD + cycle tests
06	Employees + profile lifecycle	05	Employee suite green
07	Geography: reference + sales nodes, scope ladder	05	Subtree perf test
08	Distributors + portal users	07	Isolation suite green
09	Outlets + assignments + import	08	Import dry-run E2E
10	Products: catalog + price lists	02	Price resolution tests
11	Field force: plans, stops, visits core	09	Visit lifecycle E2E
12	Route day patterns + conflict rules	11	Plan conflict tests
13	Mobile visit flow + offline sync engine	11	Sync adversarial suite
14	Sales orders + state machine + approval hooks	10, 11	Order E2E + workflow
15	Transactions + performance rollups	14	Metric formula tests
16	Targets + allocation cascade	15	Cascade sum validation
17	Marketing core: campaigns, POSM	16	Campaign state E2E
18	Trade execution + competitor capture	13, 17	Visit-linked execution
19	Inventory visibility + movements	14	Negative-stock rejection
20	Invoices, payments, outstanding	15	Aging + credit tests
21	Dashboards per Chapter 14	15-20	p95 latency target
22	Report engine + 16 catalog reports + exports	21	Export scope audit
23	Notifications: events, channels, preferences	14-20	Delivery retry tests
24	Offline hardening + conflict UX	13	Field test script pass
25	SaaS billing: plans, subscription states, limits	02	Limit enforcement tests
26	Security hardening pass + dependency scan	all	Threat checklist closed
27	Performance: indexes, rollups, cache, partitioning	21	NFR suite green
28	Production cutover: IaC, backups, monitors, runbooks	26, 27	Restore drill pass

Table 23.2 - The 28-step master build order with dependencies and gates

23.3 Module Development Protocol (for the AI agent)

1. **Inspect before writing.** Read the current schema, the affected modules and this spec's chapter for the module; write a short plan in the worklog before the first edit.
2. **Shared infrastructure changes need a blast-radius list.** Enumerate affected modules and run their regression suites; if the change alters a business rule, update the spec chapter in the same commit.
3. **One module per pass.** Land vertical slices (migration + service + API + UI + tests) rather than horizontal layers that leave the system unrunnable.
4. **Never silently change business rules.** Status vocabularies, metric formulas and permission codes are contracts; changes require a spec update note and migration of affected data.
5. **Verify the five states and isolation on every module** before marking the DoD checklist; unchecked boxes block the next build step.

CHAPTER 24

Infrastructure, DevOps & Production Readiness

The platform is cloud-agnostic: it deploys to any environment that offers containers, a managed relational database with PITR, object storage and a queue. This chapter fixes the topology, the pipeline, the observability floor and the recovery contract; concrete cloud services are chosen at implementation time against these shapes.

24.1 Deployment Topology

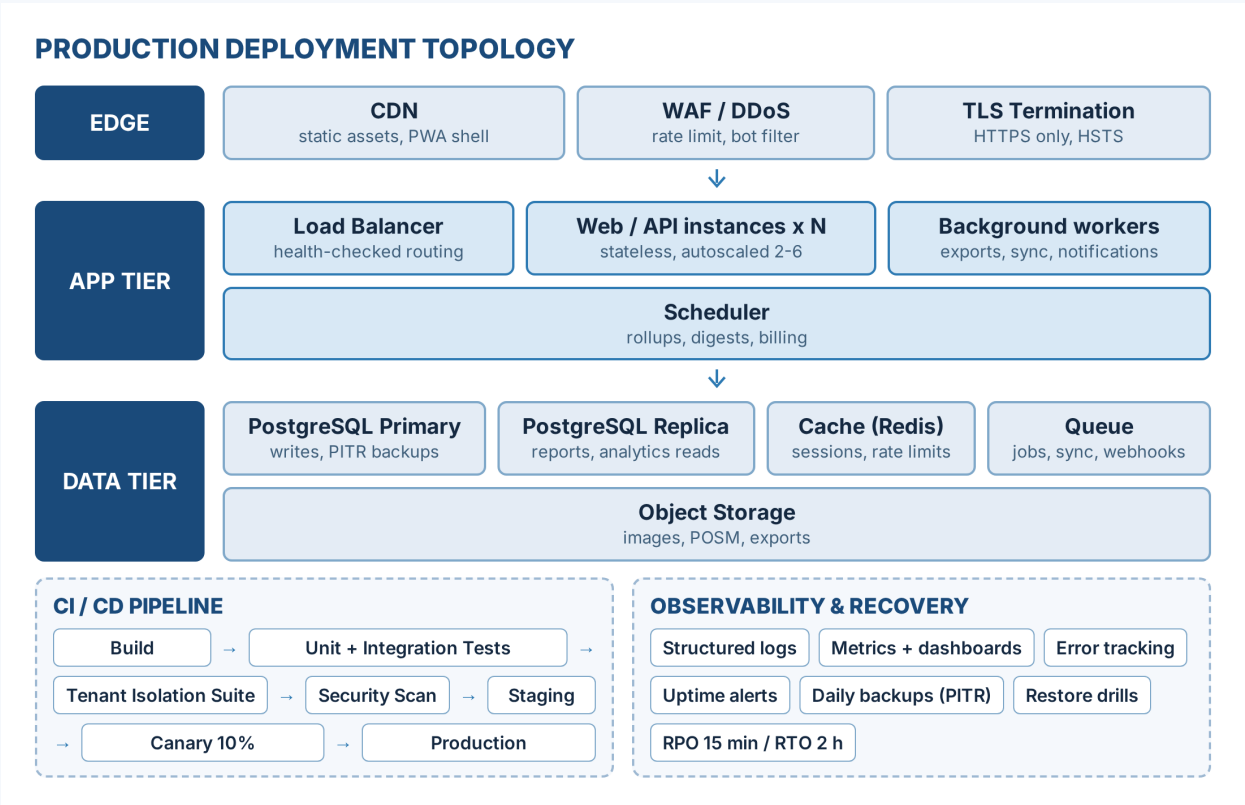


Figure 24.1 - Edge, application and data tiers with the CI/CD pipeline and observability panels

Environment	Purpose	Data policy
Development	Per-developer stacks via containers + seed	Synthetic seed only
Staging	Pre-release mirror of production topology at 1/4 scale	Anonymized refresh monthly
Production	Multi-AZ, autoscaled app tier, PITR database	Live; access audited

Table 24.1 - Environment matrix

24.2 CI/CD Pipeline

Stage	Content	Failure policy
Build	Compile, lint, unit tests, dependency scan	Block merge
Integration	Migrations up/down, API + authorization suites	Block merge
Isolation	Tenant isolation parameterized suite (Ch 22.2)	Block merge - zero tolerance
Package	Immutable container images tagged by commit	Never latest-tag
Staging deploy	Auto on green main; smoke + load smoke	Auto-rollback on smoke fail
Canary	10% traffic 30 min; error-rate and latency gates	Auto-rollback
Production	Rolling deploy; migration job runs before rollout	Expand-only migrations; contract steps for breaking changes

Table 24.2 - Pipeline stages and gates

24.3 Observability & Recovery

- Structured logs with request/tenant/trace ids; PII never logged; 30-day hot retention.
- Metrics: API latency/error rates, queue depths, sync backlog, DB health, per-tenant usage counters; dashboards per tier.
- Alerting: error-rate burn, sync backlog age, approval escalation backlog, backup freshness, certificate expiry.
- Backups: automated daily full + WAL/PITR; quarterly restore drills with timed RTO evidence; backup access restricted and audited.
- Runbooks: tenant provisioning, suspension, restore, key rotation, incident communication - each rehearsed once before cutover.

APPENDICES

Appendices

The appendices collect reference material the build agent needs at hand: the full glossary, the complete default permission catalog that doubles as the seed and the authorization test oracle, the seed data requirements, and the explicitly deferred scope so nothing deferred is accidentally forgotten or silently re-scoped.

A Appendix A - Glossary (consolidated)

The authoritative glossary is Table 2.3. Terms used in this document have exactly the meanings given there; UI copy, code identifiers and reports must use them consistently. Where a tenant customizes labels (for example calling CSRs "Rep"), only the label layer changes - code and API names remain fixed to the glossary terms.

B Appendix B - Default Permission Catalog

Permissions follow module.action. The catalog below is the seed set and the authorization test oracle; roles bundle these codes. Sub-resource permissions (sales.order.approve) imply their parent read permission.

Module	Permission codes
employee	employee.view, employee.create, employee.update, employee.status, employee.export
user	user.view, user.create, user.update, user.delete, user.roles, user.scope
role	role.view, role.manage
org	org.view, org.manage
geo	geo.view, geo.manage

Module	Permission codes
distributor	distributor.view, distributor.create, distributor.update, distributor.status, distributor.user.manage, distributor.export
outlet	outlet.view, outlet.create, outlet.update, outlet.assign, outlet.import, outlet.export
product	product.view, product.create, product.update, product.delete, product.cost.view, product.price.manage, product.import
route	route.view, route.manage
attendance	attendance.view, attendance.create, attendance.correct
visit	visit.view, visit.create, visit.update, visit.export
sales.order	sales.order.view, sales.order.create, sales.order.submit, sales.order.approve, sales.order.cancel, sales.order.fulfill, sales.order.export
sales.transaction	sales.transaction.view, sales.transaction.export
target	target.view, target.create, target.allocate, target.approve, target.achievement.view, target.export
campaign	campaign.view, campaign.create, campaign.update, campaign.approve, campaign.execution.view, campaign.export
trade	trade.execute, trade.view, trade.verify
competitor	competitor.view, competitor.create, competitor.export
posm	posm.manage, posm.view
stock	stock.view, stock.movement.create, stock.export
invoice	invoice.view, invoice.export
payment	payment.view, payment.create, payment.void
workflow	workflow.decide, workflow.manage
reports	reports.view, reports.export, reports.schedule
audit	audit.view, audit.export
tenant	tenant.settings, tenant.features.manage
platform	platform.tenant.view, platform.tenant.create, platform.tenant.suspend, platform.tenant.plan, platform.tenant.features, platform.support.grant, platform.audit.view

Table B.1 - Complete default permission catalog (seed + test oracle)

C Appendix C - Seed Data Requirements

Seed	Content
Platform	SUPER_ADMIN + PLATFORM_SUPPORT users; plans (FREE, STARTER, BUSINESS, ENTERPRISE); feature flag catalog
Permissions	All codes from Appendix B
Role templates	Full role catalog of Table 4.1 with permission bundles
Reference geography	Bangladesh divisions, districts, upazilas (extensible to other countries)

Seed	Content
Demo tenant	ABC Tobacco per Chapter 2: org hierarchy, Chattogram sales geography, 3 distributors, 150 outlets, 40 SKUs across 4 brands, 90 days of visits/orders/transactions, one active campaign, targets cascaded

Table C.1 - Seed data set (idempotent data migrations)

D Appendix D - Explicitly Deferred Scope

Deferred item	Boundary note
Full warehouse ERP (bins, batches, expiry, pick waves)	stock_movement carries batch_ref columns; module slots in later without schema shock
Native mobile apps	PWA is the v1 mobile surface; native shells are a delivery concern, API is ready
Route optimization / AI suggestions	Sequence is manual in v1; route_stop.sequence and visit history provide the training corpus
Scheme / trade-offer engine (buy 9 get 1, slabs)	free_qty and discount fields exist; automation arrives post-v1
Multi-language beyond en/bn	Locale service is centralized; adding locales is data work
Tenant-level custom branding (themes, domains)	Token-based design system makes it feasible; scheduled after billing
Data warehouse / BI extracts	Reports cover v1 needs; nightly snapshots to a warehouse are a Phase-2 addition

Table D.1 - Deferred with intent - nothing here blocks v1 acceptance